Schema-Graph-Constrained Natural Language Interface for Intermetallic Compound Exploration

Satoshi Minamoto[a,*]

[a]*National Institute for Materials Science (NIMS), 1-2-1 Sengen, Tsukuba, Ibaraki 305-0047, Japan*

Abstract

$L1_2$-type intermetallic compounds (Cu_3Au-type, including Ni_3Al-based γ' phase candidates) are important as precipitation-strengthening phases for next-generation heat-resistant superalloys; however, SQL knowledge is indispensable for cross-sectional exploration of structural, compositional, phase stability, and property data accumulated in first-principles calculation databases. In this study, we propose a **Schema Graph-constrained Text-to-SQL (T2SQL) method**.

This method consists of the following technical components: (1) a **domain-specific condition extractor** equipped with a Japanese–English bilingual materials terminology dictionary (including synonym normalization for $L1_2/\gamma'/Cu_3Au$-type expressions, a numerical condition parser, and a chemical formula parser), (2) a **Schema Graph traversal engine** that represents foreign-key (FK) relationships as a NetworkX graph and computes minimal JOIN paths using Steiner tree approximation, (3) a **multilayer SQL guard** (8-layer validation) that performs keyword blacklisting, table/column whitelisting, JOIN validation, and automatic LIMIT injection, (4) a **Schema Linker** that infers tables and columns from extracted conditions, and (5) **LLM-constrained SQL generation** using prompts with JOIN constraints.

As a validation database, 1,471 entries across 5 prototypes (393 $L1_2$-type, 636 B2-type, 355 NaCl-type, 74 NiAs-type, and 13 BiF_3-type entries, including 11 known $L1_2$ compounds such as Ni_3Al, Co_3Ti, and Al_3Sc) were stored in a normalized RDB in which composition, structure, thermodynamic stability, calculation conditions, and properties are connected by foreign keys. For 100 evaluation queries (20 Easy, 30 Medium, 30 Hard, and 20 Very Hard, with up to 4-hop JOINs), comparative experiments were conducted for five methods: (i) LLM-only (without schema information), (ii) Full Schema (all schema presented), (iii) Rule-based (without an LLM), (iv) FK-list (only FK information presented), and (v) **Proposed** (Schema Graph + entity extraction + Schema Linker + constrained LLM).

As evaluation metrics, we introduced **common-column matching** (row matching based on column names), which eliminates underestimation due to differences in SELECT columns, and **LIMIT 10,000 normalization**, which removes the effect of LIMIT truncation. The Proposed method (LLM: gpt-5.5) achieved a **syntax validity rate of 99%**, an execution success rate of 99%, and a mean execution accuracy of **70.2%**, while realizing a **table hallucination rate of 0%** (compared with 5% for LLM-only). For Medium difficulty, the execution accuracy reached 83.9%, and even for multihop queries of 2 hops or more, accuracies of 76.9% (2-hop), 71.8% (3-hop), and 45.4% (4-hop) were recorded. A repair loop for execution errors or empty results (up to two retries) recovered 3 of the 4 cases that failed in the initial generation. By limiting the LLM's field of view to the necessary tables through Schema Graph traversal, the method simultaneously eliminated unnecessary JOINs and prevented table hallucination. In an additional evaluation using 100 queries (covering 30 tables) designed by an independent materials researcher, an execution accuracy of **69.0%** was recorded, while maintaining an SQL syntax validity rate of 100.0%.

As a materials-engineering evaluation, the method achieved **complete rediscovery** (11/11) of the 11 known $L1_2$ compounds, ranking output of γ' phase candidates (a weighted composite score of stability, lattice matching, and bulk modulus; top candidates: Cu_3Nb, Ni_3Al, Cu_3Ti, and Co_3Ti; however, Cu-based candidates require experimental validation), extraction of candidates with lattice constants near that of Ni_3Al, and automatic generation of materials-design hypotheses.

Because this method is based on runtime introspection of FK relationships, it is architecturally applicable to arbitrary normalized materials RDBs (OQMD, Materials Project, AFLOW, etc.); however, domain adaptation of the terminology dictionary and Steiner tree performance for large-scale schemas require separate validation.

Keywords: Text-to-SQL, $L1_2$-type intermetallic compounds, schema graph, Steiner tree approximation, γ' phase, materials database, natural language processing, materials informatics

1. Introduction

1.1. Data-Access Barriers in AI for Science and Materials Informatics

The essential challenge of natural language interfaces for materials databases is not the syntactic generation of SQL statements itself, but rather correctly joining normalized heterogeneous materials data according to foreign-key relationships and materials-domain semantics. In this paper, targeting the exploration of $L1_2$-type intermetallic compounds (γ' phase candidates), we propose a constrained method based on Schema Graph traversal (a graph representation of FK relationships and Steiner tree approximation), and report that, in an evaluation using 1,471 entries (5 prototypes) and 100 queries, the Proposed method achieved a table hallucination rate of 0% and an execution accuracy of 70.2%.

AI for Science (AI4S)—the systematic application of artificial intelligence to accelerate scientific discovery—is transforming materials research [1, 2]. AI4S envisions integrated workflows in which AI agents autonomously generate hypotheses, retrieve relevant data from large-scale databases, design experiments and simulations, and interpret results. In this vision, **natural language access to structured materials data** is not merely a convenience but a prerequisite: if AI agents (or the researchers directing them) cannot efficiently query normalized relational databases, the entire AI4S pipeline becomes bottlenecked at the data retrieval stage.

With the rapid development of high-throughput first-principles calculations, multiple large-scale materials databases have been constructed: the Open Quantum Materials Database (OQMD, approximately one million entries) [3, 4], Materials Project (over 150,000 entries) [5], AFLOW (over 3.5 million entries) [6], and NOMAD [7]. These databases are indispensable for computational materials exploration, screening of candidate phases [8, 9], and validation of experimental observations. Meanwhile, as efforts toward structuring and knowledge representation of materials data, Ishii et al. have constructed an ontology and RDF representation of the legacy superconductivity database SuperCon [10] and realized machine-readable conversion of the polymer database PoLyInfo [11], demonstrating that semantic structuring of schemas is a prerequisite for utilizing materials data.

However, access to data stored in normalized relational schemas (multiple tables connected by foreign keys) requires SQL knowledge. What is fundamentally important in materials engineering is not the collection of massive amounts of data but **the accurate combination of small amounts of heterogeneous data**: composition, crystal structure, thermodynamic stability, calculation conditions, and property values are normalized into separate tables, and only by correctly JOINing them along foreign keys (FKs) can meaningful materials information be obtained. For example, retrieving "lattice constants of stable B2 compounds containing Fe" requires a four-table JOIN with WHERE conditions on elements, prototypes, and thermodynamic stability—an easy task for a database engineer, but a nontrivial operation for a materials researcher. Inaccurate JOINs (unnecessary table joins or missing join conditions) return erroneous combinations of data and pose a serious risk of misleading decisions in materials exploration and screening.

REST APIs and GraphQL endpoints provided by many materials databases partially alleviate this barrier, but fundamental constraints common to RDBs remain: (a) dependence on predefined filter syntax (making ad hoc compound conditions and aggregation difficult), (b) risk of API incompatibility when schemas are updated, and (c) the frequent absence of APIs for in-house databases or independently extended schemas. Because the T2SQL approach analyzes schema structures at runtime, it is in principle applicable to arbitrary normalized RDBs regardless of the presence or absence of an API (although preparation of terminology dictionaries and performance validation for large-scale schemas are separately required).

In the context of AI4S, this data-access barrier is particularly serious: autonomous AI agents performing materials screening or property prediction must programmatically issue complex compound-condition database queries. A natural language interface that reliably generates correct SQL enables both human researchers and AI agents to interact with materials databases without expertise in database engineering, thereby removing a key bottleneck in AI4S workflows.

1.2. Technical Trends in Text-to-SQL

Text-to-SQL (T2SQL) research has advanced rapidly through large-scale benchmarks: Spider [12] (10,181 questions, 200 databases, 138 domains) and BIRD [13] (12,751 questions, 95 databases, 33.4 GB). Recent LLM-based methods have achieved high accuracy: DAIL-SQL [14] reported an execution accuracy of 86.6% on Spider through skeleton-based few-shot example selection, DIN-SQL [15] achieved 85.3% through decomposed in-context learning, and XiYan-SQL [16] reported further accuracy improvements using a multi-generator ensemble. Hong et al. [17] provide a comprehensive survey highlighting the dominant role of schema linking, and Maamari et al. [18] argue that explicit schema linking may become unnecessary with advanced LLMs, although this remains under debate.

Methods that explicitly use schema structures as graphs have also developed. RAT-SQL [19] pretrains inter-table relationships through Relation-Aware Schema Encoding, and ShadowGNN [20] encodes schema graphs using a graph projection neural network. RESDSQL [21] separates schema linking and skeleton parsing, and Bridge [22] bridges text and tabular data. More recently, SchemaGraphSQL [23] proposed large-scale database schema link-

*Corresponding author
Email address: minamoto.satoshi@nims.go.jp (Satoshi Minamoto)

ing using path-finding algorithms, and GraST-SQL [24] addresses scaling of schema filtering using functional dependency graphs. McMillan [25] evaluates the impact of structured context engineering on schema accuracy in multi-file agent systems.

However, **all existing T2SQL benchmarks target general-purpose domains** (e-commerce, healthcare, and education), and no prior study has addressed challenges specific to materials science databases: domain-specific terminology (Strukturbericht symbols and prototype names), bilingual queries (Japanese and English element names), normalization of composition tables (one row per element per compound), and thermodynamic stability filters ($E_{\mathrm{hull}} \leq 0.001$ eV/atom).

1.3. NLP in Materials Informatics

Applications of NLP in materials science are expanding: MatSci-NLP [26] benchmarks seven NLP tasks for materials texts, LLAMAT [27] improves information extraction through continued pretraining of LLaMA, the Materials Knowledge Navigation Agent (MKNA) [28] converts natural language intents into database search and structure-generation actions, and OptiMat Alloys [29] provides an LLM-driven interactive alloy exploration tool. Materials Project has begun providing MCP-based LLM integration [30].

Although these studies address information extraction and agent-based database interaction, none targets **schema-aware SQL generation over normalized relational databases**—in particular, automatic JOIN path discovery through foreign-key graph traversal. This gap is the focus of the present method.

1.4. Relationship to Ontology and Knowledge Graph Approaches

For structured access to materials data, approaches based on ontologies and knowledge graphs (KGs) also constitute a prominent lineage. Ishii et al. reconstructed the superconducting materials database SuperCon as an RDF ontology and realized semantic search through a SPARQL endpoint [10]. The same group also performed similar machine-readable conversion for polymer data in PoLyInfo, enabling cross-domain data integration through alignment with the BFO upper ontology [11]. In Europe, the Elementary Multiperspective Material Ontology (EMMO) has been developed as a standard ontology for materials modeling, providing guarantees of semantic consistency based on OWL reasoning [31].

In general-purpose domains, Sequeda & Allemang [32] converted enterprise SQL databases into knowledge graphs and showed that Text-to-SPARQL improves accuracy from 16% to 54% compared with Text-to-SQL. Furthermore, query validation using ontology semantic constraints (OBQC) improved accuracy to 72% [33].

The present method shares with these ontology approaches the objective of **query quality assurance through structural constraints**. Domain/range constraints in OWL properties of ontologies functionally correspond to inter-table constraints imposed by FK relationships in RDBs, and SPARQL query validation by OBQC plays a role equivalent to that of SQLGuard in the present method (column whitelisting and FK-consistent JOIN validation). However, the application contexts of the two approaches differ. Ontology approaches presuppose data conversion into RDF triple stores and have superior expressive power for semantic transitivity through OWL reasoning (e.g., "NiAl is $L1_2$-type" → "NiAl is an intermetallic compound") and interoperability among heterogeneous databases. In contrast, large-scale materials databases such as OQMD [3], Materials Project [5], and AFLOW [6] are already operated using relational schemas, and the present method adopts a design that **leverages these existing RDB infrastructures as they are**. Graph traversal of FK relationships (Steiner tree approximation) provides a lightweight alternative that guarantees the correctness of JOIN paths without relying on ontology reasoning.

The two approaches are not mutually exclusive but complementary. In the future, integrating semantic constraints defined by ontologies into Schema Graph traversal heuristics may enable JOIN path selection based not only on FK relationships but also on domain knowledge.

1.5. Objectives and Novelty

The contributions of this study are the following five points:

1. **A Schema Graph-constrained T2SQL method specialized for $L1_2$-type compound exploration**: by combining a Japanese–English bilingual materials terminology dictionary with FK relationship graph traversal, the method realizes automatic JOIN path computation on normalized materials databases, including synonym normalization for expressions such as $L1_2/\gamma'/Cu_3Au$-type. Schema Graph constraints achieve a **table hallucination rate of 0%**, eliminating hallucinations in the LLM-only method while achieving an **execution accuracy of 70.2%** by eliminating unnecessary JOINs.
2. **Quantitative evaluation through comparison with five method baselines**: five methods—LLM-only, Full Schema, Rule-based, FK-list, and Proposed—are compared using 100 queries (Easy/Medium/Hard/Very Hard, up to 4-hop), systematically evaluating the effect of how schema information is presented on SQL quality.
3. **Safety assurance through a multilayer SQL guard**: keyword blacklisting, table/column whitelisting, JOIN validation, and automatic LIMIT injection are integrated to prevent execution of invalid SQL.
4. **Materials-engineering evaluation**: the method achieves complete rediscovery (11/11) of the 11 known $L1_2$ compounds, ranking of γ' phase candidates, extraction of candidates with lattice constants near that

of Ni_3Al, and automatic generation of materials-design hypotheses.

5. **Multihop query evaluation**: for 47 multihop queries requiring 2–4 table JOINs, we report the execution accuracy of the Proposed method and comparisons with each baseline.

The overall architecture of the present method is shown in Fig. 1 (overview) and Fig. 2 (detailed configuration).

2. Design Requirements for Materials Text-to-SQL

The essential challenge of a natural-language interface for materials databases is not the syntactic generation of SQL statements itself, but rather **correctly joining normalized heterogeneous materials data according to foreign-key relationships and materials-domain semantics**. When a general-purpose Text-to-SQL system is directly applied to a materials database, the following materials-specific requirements are not satisfied, leading to a substantial degradation in accuracy.

1. **Accurate JOINs of normalized composition tables**: Because each element in a multicomponent compound is stored as an independent row, searches for compounds containing *both* "Ni and Al" require a SELF-JOIN or INTERSECT. General-purpose LLMs frequently make errors in this pattern.
2. **Disambiguation of materials terminology**: Whether "NiAl" denotes a specific compound (B2-NiAl) or a contained-element condition is context-dependent, and whether "stable" means $E_{\mathrm{hull}} \leq 0.001$ eV/atom or $\Delta G_f < 0$ is schema-dependent. Without a domain dictionary, column mapping becomes indeterminate.
3. **Vocabulary normalization of prototypes and space groups**: $L1_2$, $AuCu_3$ type, and Cu_3Au type refer to the same prototype, but their notations are diverse, requiring synonym expansion in both Japanese and English.
4. **Computation of minimal JOIN paths along FKs**: In a schema with approximately 30 tables, candidate JOIN paths proliferate, and joining unnecessary tables reduces LLM accuracy. Extracting a minimal subgraph via a Steiner tree approximation is indispensable.
5. **Interpretation of units and thresholds in numerical conditions**: Whether "a large band gap" means > 1.0 eV or > 2.0 eV, and whether "stable" means $E_{\mathrm{hull}} \leq 0.001$ or ≤ 0.1 requires judgment based on materials-domain knowledge.
6. **Rejection of out-of-scope questions**: Questions about the optimization of VASP calculation parameters or DFT workflows are not SQL-answerable, and forcing SQL generation returns meaningless results.
7. **Guarantees for safe execution**: Exclusion of non-SELECT SQL (INSERT/UPDATE/DELETE), prevention of unrestricted result sets (automatic LIMIT injection), and injection prevention using table and column whitelists.

These requirements are materials-database-specific challenges that are not included as evaluation targets in general-purpose Text-to-SQL benchmarks (Spider, WikiSQL, etc.). The proposed method was designed as an integrated framework that addresses all seven requirements above.

3. Methods

This section details each technical component. Figure 2 shows the overall pipeline architecture of the system.

3.1. Database Schema Design for Materials Data

3.1.1. Normalized Schema Design for OQMD Data

OQMD systematically stores composition, structure, and thermodynamic stability, and is a representative example of the schema complexity typical of materials databases (many-to-many relationships, normalized composition tables, and separation of structural parameters). Flat JSON records obtained from the API are normalized into a relational schema (Table 1). The design follows Boyce-Codd normal form (BCNF) and is based on the following materials-domain-specific considerations.

Separation of the Composition Table. A single compound may contain multiple elements (e.g., $Fe_3Al \rightarrow$ Fe and Al). By storing each element as a separate row in the `composition` table, flexible element-based queries (AND, OR, NOT) are enabled without string parsing [34]. However, this normalization introduces an important complexity: a query for "compounds containing both Ni and Al" cannot be expressed by a simple `WHERE element = 'Ni' AND element = 'Al'` (because no single row can satisfy both conditions simultaneously, 0 rows are returned). Instead, `EXISTS` subqueries are required (see Section 3.3.4). This design also enables searches that do not depend on the notation order of chemical formulas (e.g., Ni_3Al and $AlNi_3$ are the same compound, but notation conventions differ across databases). Because elements and atomic fractions are stored as separate rows, the same compound can be uniquely identified by element-level queries rather than by string comparison of the entire formula.

Separation of Structure/Phase Stability/Calculation. Crystal structure information (prototype, lattice constants), thermodynamic stability (energy above hull), and calculation metadata (DFT functional) are stored in separate tables, enabling independent updates and extensions.

Entity-Attribute-Value (EAV) Pattern for Calculated Properties. The `calculated_property` table uses the EAV pattern (`property_name`, `value`, `unit`), allowing arbitrary calculated properties (bulk modulus, shear modulus, etc.) to be accommodated without schema changes. In the EAV design, in addition to the column whitelist, a list of permitted values for `property_name` (`bulk_modulus`, `shear_modulus`, etc.) is maintained, enabling detection when the LLM generates a nonexistent property name.

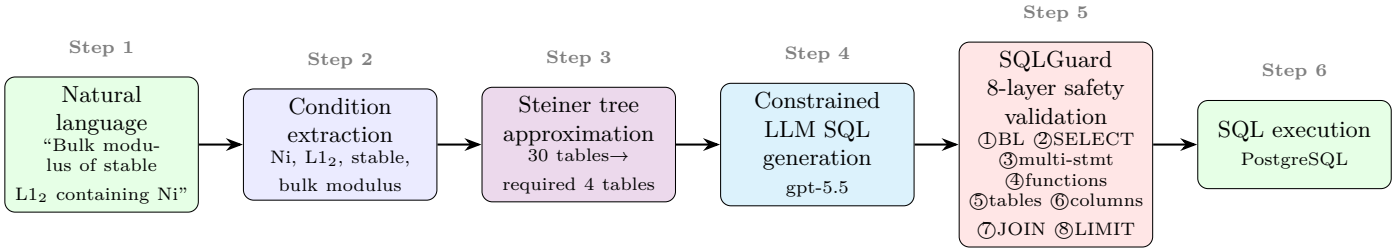

Figure 1: Overview of the Step 3 pipeline. Starting from a natural language query and condition extraction (Step 2), Steiner tree approximation (Step 3) extracts the minimal necessary subgraph from 30 tables. The extracted tables, columns, and JOIN conditions are injected into the LLM as YAML (Step 4), and safe SQLFN is executed after passing through the 8-layer SQLGuard (Step 5). Because the LLM's field of view is limited to the necessary tables, unnecessary JOINs and table hallucinations are eliminated in principle.

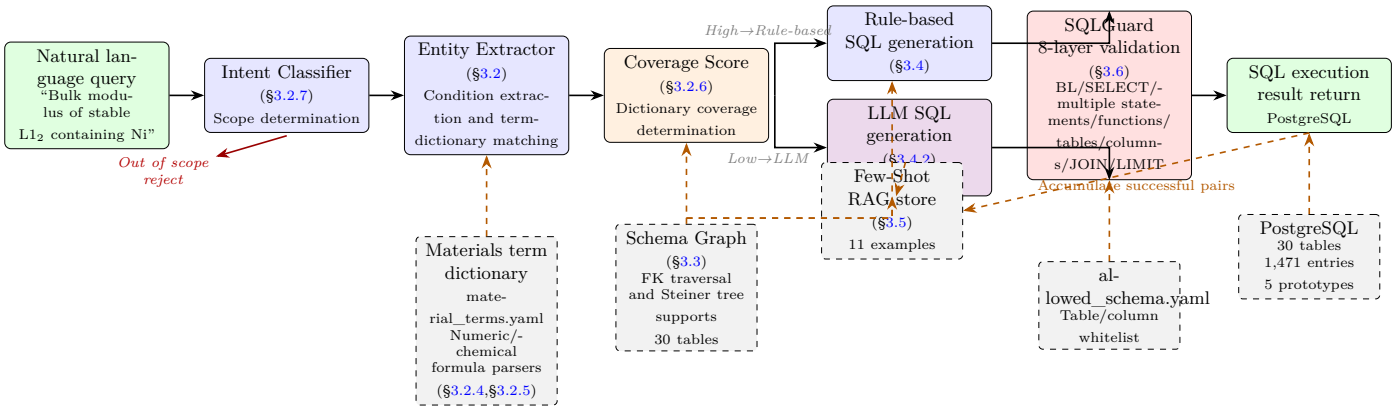

Figure 2: Overall configuration of the system architecture. Solid arrows indicate the main data flow, and dashed arrows (orange) indicate data references and feedback loops. Based on the Coverage Score, the pipeline branches to either the Rule-based path (high coverage) or the LLM path (low coverage), and in both cases, safe SQL is generated and executed through JOIN-path constraints imposed by the Schema Graph (FK traversal over 30 tables) and SQLGuard 8-layer validation. When an execution error or an empty result is detected, a repair loop (up to 2 attempts) is activated, requesting the LLM to perform a repair with error diagnostic information. Successful query pairs are accumulated in the Few-Shot RAG store and used for subsequent LLM generation.

Table 1: Overview of the database schema.

Table name	Primary key	Foreign key	Main columns
material_entry	entry_id	—	formula, reduced_formula, chemical_system
composition	composition_id	entry_id	element, atomic_fraction
structure	structure_id	entry_id	prototype, strukturbericht, lattice_a, space_group
phase_stability	stability_id	entry_id	formation_energy_per_atom, energy_above_hull, band_gap
calculation	calculation_id	entry_id	method, functional
calculated_property	property_id	calculation_id	property_name, value, unit
prototype_definition	prototype_id	—	prototype_name, strukturbericht, description

Note on the Generality of ER Design. Although OQMD is internally operated on a relational database (Django ORM + PostgreSQL), in this study we newly designed an ER diagram that is easy to traverse as a NetworkX graph and YAML schema, using the flat JSON data obtained from the API. This design decision generalizes the applicability of the proposed method to the following two scenarios: (1) directly extracting and traversing schema information (FK constraints) from an existing RDB, and (2) constructing and traversing a new purpose-specific ER diagram from flat data. In either case, if FK relationships can be represented as a graph, JOIN-path optimization by Schema Graph traversal is applicable.

3.1.2. XML/YAML Representation for RAG Context Injection

The schema structure is serialized as YAML (`allowed_schema.yaml`) and used for a dual purpose: (a) the table/column whitelist for the SQL guard (Section 3.6), and (b) RAG context for LLM prompts—the schema YAML is injected into the system message, allowing the LLM to understand the available tables, columns, and types. This approach is similar to schema serialization techniques in general-purpose T2SQL [14], but is specialized for the domain-specific column semantics of materials databases (e.g., `energy_above_hull` represents E_{hull}).

3.2. Condition Extractor with a Materials Terminology Dictionary

The condition extractor uses a YAML-based Japanese–English bilingual terminology dictionary (`material_terms.yaml`) to convert natural-language queries into structured condition dictionaries.

3.2.1. Dictionary Structure

The dictionary contains four categories:

1. **Prototype aliases**: mappings between prototype names and their variants.
   Example: `B2` ↔ [`B2`, `CsCl`, `CsCl` , `B2`]
   `L12` ↔ [L1_2, `L12`, `AuCu3`, Cu_3Au , γ']
2. **Element mapping**: element symbols ↔ Japanese and English names.
   Example: `Fe` ↔ [`Fe`, , `iron`]
3. **Stability terms**: natural language → numerical thresholds.
   "stable"/ "stable" → `energy_above_hull` ≤ 0.001 eV/atom
   "metastable"/ "metastable" → 0.001 < `energy_above_hull` ≤ 0.05 eV/atom
   "stable or metastable" → `energy_above_hull` ≤ 0.05 eV/atom
   In SQL conditions, floating-point precision is taken into account, and range conditions (`<= 0.001`) are used rather than equality comparisons (`= 0`).
4. **Property terms**: mappings from property descriptions to table.column references.
   "lattice constant"/ "lattice constant" → `structure.lattice_a`
   "formation energy"/ "formation energy" → `phase_stability.formation_energy_per_atom`

3.2.2. Unicode Normalization and Pattern Matching

Unicode subscripts (`U+2080`–`U+2089`, namely $_{0}{1}{2}\cdots{9}$) are normalized to ASCII digits before matching. Prototype names are normalized to ASCII notation (`L12`, `B2`, etc.) at input, and variations in natural-language notation (`L1_2`, L1_2, γ', Cu_3Au) are mapped by the dictionary to the internal DB value (`L12`). Element-symbol recognition uses word-boundary regular expressions, correctly distinguishing an independent "Fe" from the substring in "FeCoNi".

3.2.3. Output Format

The extractor returns a structured dictionary:

Listing 1: Example of condition extraction.

```
# Input: "Show stable B2 compounds containing Fe"
# Output:
{
  "prototype": "B2",
  "contains_elements": ["Fe"],
  "stability": "stable",
  "sort_by": None,
  "sort_order": None
}
```

3.2.4. Numeric Condition Parser

The condition extractor includes a regular-expression-based numeric condition parser that converts numerical comparison expressions in natural language into structured WHERE-clause conditions. The five supported pattern types are shown in Table 2.

Table 2: Supported patterns of the numeric condition parser.

Pattern type	Input example	Output
Symbolic comparison	`band_gap > 1.0 eV`	`WHERE band_gap > 1`
Japanese postfix	formation energy is 0.5 or greater	`WHERE ... >= 0.5`
Japanese "than"	lattice constant is greater than 3.0	`WHERE lattice_a >`
Sign test	formation energy is negative	`WHERE ... < 0`
Range specification	band gap 0.5–2.0 eV	`WHERE ... BETWEEN 0.5 AND 2.0`

The mapping from property names to the corresponding columns is managed by an internal dictionary (`_PROPERTY_COLUMN_MAP`), which performs conversions such as "lattice constant" → `structure.lattice_a` and "formation energy" → `phase_stability.formation_energy_per_atom`. Unit conversion (eV, Å, GPa, etc.) is also built in, but in the current validation DB, property values are normalized to standard units (eV/atom, Å, GPa) at ingestion, and all conversion factors are 1.0. For generalization, a mechanism is required to generate WHERE conditions after converting input units to standard units.

Floating-Point Comparisons and Tolerances. Because DFT-calculated values are subject to floating-point precision limitations, strict equality comparisons (e.g., `lattice_a = 3.572`) are avoided, and range conditions (`BETWEEN 3.571 AND 3.573`) are preferred. For natural-language expressions such as "near" and "comparable," tolerances defined for each property type (lattice constant: ± 0.03 Å, E_{hull}: ± 0.001 eV/atom) are applied from the dictionary.

Handling NULL Values. Entries for which property values have not been calculated (`NULL`) are automatically excluded from the results of numerical comparison queries (e.g., "high bulk modulus"). An `IS NOT NULL` condition is implicitly added during SQL generation, preventing missing values from distorting rankings or statistics.

3.2.5. Chemical Formula Parser and Ambiguity Handling

The chemical formula parser detects chemical-formula tokens in queries (e.g., `Ni3Al`, `NiAl`, `Fe2CoAl`) and classifies them into the following three interpretations:

- **exact_formula**: when the stoichiometric ratio is explicitly specified (e.g., `Ni3Al` → `formula = 'Ni3Al'`). It generates an exact-match search against the chemical-formula column of the composition table.
- **formula_or_reduced_formula**: when the stoichiometric ratio is omitted but the token is used as a standalone chemical formula (e.g., "B2 entries for NiAl" → `reduced_formula = 'AlNi'`). It preferentially generates a match search against the `reduced_formula` column.
- **contains_elements**: when used in inclusion expressions such as "containing X" or "compounds containing X and Y" (e.g., "containing Ni and Al" → element Ni **AND** element Al). It generates EXISTS subqueries (multi-element AND) against the composition table.

The decision is context-dependent: when a chemical-formula token independently co-occurs with a prototype name (B2, $L1_2$, etc.) or with words such as "entry" and "property," `formula_or_reduced_formula` is prioritized; when it co-occurs with inclusion expressions such as "contains," "both," or "contains," it is processed as `contains_elements`. This context-dependent decision is a design choice based on domain knowledge: when materials researchers say "NiAl," they often mean the specific compound B2-NiAl, whereas "compounds containing Ni and Al" indicates an inclusion search.

3.2.6. Coverage Score: Detection of Unrecognized Vocabulary

The Coverage Score quantifies, on $[0, 1]$, the fraction of semantic tokens contained in a natural-language query that are recognized by the terminology dictionary, element dictionary, or numerical patterns. This metric was introduced to prevent Silent Constraint Dropping (silently discarding unregistered vocabulary).

Computation procedure:

1. Tokenize the query using regular expressions and remove stop words (particles, suffixes, etc.)
2. Match each token against dictionary aliases (prototypes, elements, properties, stability)
3. Determine whether element symbols are known or unknown (registered in the dictionary or not)
4. Coverage = number of recognized tokens/number of all semanti

Based on the Coverage Score, the system branches into three levels of action:

- $\geq$ 0.8 (and no unknown elements) → `execute_rule_based`: safely execute in rule-based mode
- $\geq$ 0.5 (or with unknown elements) → `fallback_to_llm`: escalate to LLM mode
- < 0.5 → `clarification_required`: request clarification from the user

With this mechanism, for a query such as "compounds containing Xe," if Xe (xenon) is not registered in the dictionary, the Coverage Score decreases, and an LLM fallback or clarification request is triggered rather than generating SQL while ignoring the condition.

3.2.7. Intent Classifier: Preemptive Exclusion of Out-of-Scope Questions

In the evaluation experiments, we found that the LLM generated SQL for questions about VASP workflows (out-of-scope accuracy 0%). To address this problem, we introduced a rule-based Intent Classifier **upstream** of the SQL generation pipeline.

The Intent Classifier classifies queries into the following five categories:

1. **db_query**: answerable using the materials database
2. **out_of_scope**: VASP/DFT workflows, computational methods, or general knowledge
3. **ambiguous**: clarification is required
4. **unsafe**: malicious intent such as SQL injection
5. **greeting**: greetings or small talk

The classification algorithm compares the number of regular-expression matches for 20 VASP/DFT workflow patterns (file names such as INCAR, KPOINTS, and POTCAR; parameter names such as SCF convergence and ENCUT; phonon calculations, Wannier90, Bader analysis, etc.) and 9 DB-specific patterns (compound search, stability filters, property conditions, etc.). If only VASP patterns are detected, the query is rejected as out_of_scope; if DB patterns are dominant, the query is passed to the pipeline as db_query.

3.3. Schema Graph Traversal Engine

This is the central algorithmic contribution of the present method. This engine represents the database schema as a graph and automatically computes the optimal JOIN paths.

3.3.1. Graph Construction from PostgreSQL Metadata

This implementation targets PostgreSQL. Although the architecture is portable to other RDBMSs, metadata acquisition SQL, LIMIT syntax, type information, and privilege design require DBMS-specific adjustments.

Foreign-key relationships are dynamically extracted from PostgreSQL's `information_schema`. In the validation schema of this study, we targeted single-column FKs, but for composite foreign keys (`(a, b) REFERENCES parent(a, b)`) or cases spanning multiple schemas, extension to extraction using `pg_catalog` is required:

Listing 2: FK relationship extraction query.

```
SELECT tc.table_name AS source_table,
       kcu.column_name AS source_column,
       ccu.table_name AS target_table,
       ccu.column_name AS target_column
  FROM information_schema.table_constraints tc
  JOIN information_schema.key_column_usage kcu
    ON tc.constraint_name = kcu.constraint_name
  JOIN information_schema.constraint_column_usage
      ccu
    ON ccu.constraint_name = tc.constraint_name
 WHERE tc.constraint_type = 'FOREIGN KEY'
   AND tc.table_schema = 'public';
```

The results are stored as an undirected graph $G = (V, E)$ in NetworkX [35]. Here, $V = \{\text{table names}\}$ and $E = \{\text{FK relationships}\}$. An undirected graph is used for path search, but each edge retains the FK direction (referencing table and column, referenced table and column) as attributes, and this directed metadata is used when generating JOIN clauses. Because FK metadata is read from PostgreSQL at runtime, the FK path-search component can track schema changes. However, the correspondence between natural-language vocabulary and column semantics (the terminology dictionary), the list of allowed values for `property_name`, and Few-Shot examples require separate updates.

3.3.2. Minimum-Covering JOIN Path: Steiner Tree Approximation

Given the set of *required tables* $R \subseteq V$ determined from the output of the condition extractor, a minimum subgraph of G that connects all tables in R is required. This is the Steiner tree problem in graphs [36], which is generally NP-hard.

Although the computational cost is small for the 30-table validation DB in this study, the proposed method assumes an extended schema with more than 100 tables and formulates the task as a minimum-subgraph problem that connects the required-table set. For the 30-table scale, we adopt a greedy heuristic (Algorithm 1) that gives the exact solution on tree-structured graphs:

Computational Complexity. For a graph with $|V| = n$ nodes and $|R| = k$ required tables, the algorithm performs $O(k^2)$ shortest-path computations, each of which is $O(n + |E|)$ (BFS). Even at the 30-table scale ($n = 30$, $k \leq 5$), it completes on the order of milliseconds.

Algorithm 1 Minimum-covering JOIN subgraph (Steiner tree approximation).

Require: FK relationship graph $G = (V, E)$, required-table set R
Ensure: JOIN path list P

$P \leftarrow \emptyset$, covered $\leftarrow \emptyset$
for all $(s, t) \in \binom{R}{2}$ **do**
  if $s \in$ covered $\wedge\ t \in$ covered **then**
    continue ▷ Both endpoints are already connected
  end if
  $p \leftarrow \text{ShortestPath}(G, s, t)$ ▷ BFS on the FK graph
  if $p \neq \emptyset$ **then**
    $P \leftarrow P \cup \{p\}$
    covered $\leftarrow$ covered $\cup$ nodes(p)
  end if
end for
for all $r \in R \setminus$ covered **do** ▷ Handling isolated required tables
  $c \leftarrow$ nearest node in covered
  $P \leftarrow P \cup \{\text{ShortestPath}(G, r, c)\}$
end for
return P

Correctness on Tree-Structured FK Graphs. When G is a tree (in this schema, `material_entry` is the hub), the Steiner tree is unique, and the greedy algorithm finds it exactly. In schemas with FK cycles (e.g., self-referential tables), the approximation may include unnecessary edges, but in SQL generation this does not affect correctness if DISTINCT is applied.

3.3.3. JOIN Path → SQL FROM/JOIN Clause Generation

Each path in P is converted into SQL JOIN clauses. FK metadata provides the join column names:

Listing 3: JOIN clause generation from paths.

```
# Path: [material_entry, structure]
# FK: structure.entry_id -> material_entry.entry_id
# Generated:
#   FROM material_entry m
#   JOIN structure s ON s.entry_id = m.entry_id
```

Table aliases are automatically assigned from the initial letters of table names (e.g., `material_entry`→`m`, `composition`→`c`). When initial letters collide, as in `structure` and `space_group`, subsequent letters are added to resolve the collision (`s`, `sg`). When the same table appears multiple times (e.g., multi-element AND queries), unique aliases such as `c1` and `c2` are assigned in order of appearance, and column references are also validated in the namespace after alias resolution.

3.3.4. Multi-Element AND Queries: EXISTS Subquery Pattern

There is an important materials-specific issue arising from normalization of the composition table. A query for "compounds containing both Ni and Al" requires the following:

Listing 4: Multi-element AND query using EXISTS subqueries.

```
SELECT DISTINCT m.entry_id, m.formula
FROM material_entry m
WHERE EXISTS (
    SELECT 1 FROM composition
    WHERE entry_id = m.entry_id AND element = 'Ni')
  AND EXISTS (
    SELECT 1 FROM composition
    WHERE entry_id = m.entry_id AND element = 'Al')
LIMIT 10000;
```

The Schema Graph engine detects when multiple elements are specified and automatically generates EXISTS subqueries instead of direct JOINs. Without this pattern (the naive approach), `WHERE c.element = 'Ni' AND c.element = 'Al'` always returns 0 rows because a single composition row cannot simultaneously satisfy both conditions. This is the **most important failure mode** prevented by the Schema Graph method.

The EXISTS pattern is applied only when an AND intent is detected. When OR indicators such as "Ni or Al" or "containing either" are detected, `WHERE c.element IN ('Ni', 'Al')` is generated, and the EXISTS pattern is not applied. The condition extractor (Section **??**) classifies AND/OR intent from conjunctions and particles and selects the appropriate SQL structure.

3.3.5. Comparison: Naive vs. Schema Graph (Concrete Example)

Table 3 shows the difference for "Show B2 compounds containing Fe":

Table 3: SQL generation comparison: Naive vs. Schema Graph for "B2 compounds containing Fe".

Aspect	Naive (Level 0)	Schema Graph (Level 1)
JOIN tables	Always all tables	Only required tables
Number of JOINs	For all tables	Minimal
DISTINCT	None → duplicate rows	Present
LIMIT	None → unbounded	Present (10,000)
Safety validation	None	Complete (keywords + table WL)
Multi-element AND	Incorrect (0 rows)	Correct (EXISTS)

3.4. SQL Generation: Rule-based Mode and LLM Mode

The system supports two SQL generation modes, both operating within Schema Graph constraints.

3.4.1. Rule-based Mode (Level 1)

A deterministic template-based generator constructs SQL from the extracted conditions and the Schema Graph JOIN paths. No external API is required, and the latency is 32–60 ms (Section 5.3.1).

Extensions in the Revised Version. In the previous manuscript, the Rule-based mode could not generate numerical-comparison WHERE clauses; in the revised version, the numerical condition parser (Section 3.2.4) and the chemical formula parser (Section 3.2.5) were integrated, making it possible to handle conditions such as `band_gap > 1.0`, `formation_energy < 0`, and `Ni3Al` (exact match) within the Rule-based mode.

Remaining Limitations. The following query types cannot be handled by the Rule-based mode and are automatically routed, by the Coverage Score (Section 3.2.6) or the Intent Classifier (Section 3.2.7), to LLM fallback, a clarification request, or out-of-scope rejection:

- Negated conditions ("compounds that do not contain Fe")
- Free-form queries containing vocabulary not registered in the dictionary
- VASP/DFT workflow questions (detected and rejected by the Intent Classifier)
- Complex aggregation (GROUP BY + HAVING, etc.)

3.4.2. LLM Mode with Few-Shot RAG (Level 2)

When the dictionary coverage of the Rule-based mode is insufficient (unregistered elements, numerical conditions, complex expressions, etc.), the system falls back to LLM-based generation. The LLM (gpt-5.5 in this experiment; see Supplementary Section S7 for details of the API settings) receives the following structured prompt:

1. **System message**: Task description + schema YAML (table names, column names, types, FK relationships)
2. **Few-Shot examples**: Top-k similar NL–SQL pairs from the RAG store (Section 3.5)
3. **User message**: Natural-language query
4. **Constraints**: "Generate only SELECT statements", "Include LIMIT 10000", "Use only tables listed in the schema"

The generated SQL is validated by the SQL guard (Section 3.6) before execution.

3.5. Few-Shot RAG Store (SQL-as-Few-Shot-Examples)

3.5.1. Architecture

The Few-Shot store implements a RAG [37] pattern specialized for T2SQL. Since GPT-3 [38], task-agnostic accuracy improvements through in-context presentation of a small number of examples have been widely recognized; in

this system, we apply this to materials T2SQL through dynamic retrieval of domain-specific examples:

1. **Storage**: When a query successfully returns results, the (NL query, SQL, conditions, row count, source) tuple is added to a JSON store.
2. **Retrieval**: For a new query, TF-IDF [39] cosine similarity with all stored NL queries is computed, and the top k entries (default $k = 3$) are returned.
3. **Injection**: The retrieved examples are formatted as "`Question: ... SQL: ...`" blocks and placed at the beginning of the LLM prompt.

3.5.2. TF-IDF Tokenization for Materials Queries

Standard NLP tokenizers split chemical formulas and element symbols incorrectly, making them unsuitable for materials queries. This tokenizer uses a composite regular-expression pattern:

Listing 5: TF-IDF tokenizer for materials queries.

```
# Tokenization patterns (in priority order):
# 1. Strukturbericht symbols: L12, B2, A15, D0_3,
    ...
# 2. Chemical formulas: Ni3Al, Fe2O3, ...
# 3. Chemical symbols: Fe, Ni, Al, ...
# 4. Japanese strings: [\u3040-\u9fff]+
# 5. English words: [a-z]+
# 6. Numbers: \d+
PROTO_PAT = r'[A-Za-z]\d+[_]?\d*'   # L12, B2, A15
FORMULA_PAT = r'[A-Z][a-z]?\d*'      # Ni3, Al, Fe2
tokens = re.findall(
  PROTO_PAT + r'|' + FORMULA_PAT
  + r'|[\u3040-\u9fff]+|[a-z]+|\d+',
  query   # Match while preserving case, then
      finally convert to lowercase
)
tokens = [t.lower() for t in tokens]
```

IDF is computed as $\mathrm{IDF}(t) = \log(N/n_t)$. Here, N is the total number of stored examples, and n_t is the number of examples containing term t. Ranking is performed by the cosine similarity between the TF-IDF vectors of the query and the stored examples.

3.5.3. Extraction of Seed Examples from Research Papers

To bootstrap the Few-Shot store without manual curation, we implemented automatic seed-example extraction from LaTeX manuscripts. The extractor scans for the following patterns:

- B2-type pattern: `B2[-\s]type` + chemical formula → {prototype: B2, elements: extracted}
- $L1_2$-type pattern: `L1_2` / `L12` + chemical formula → {prototype: $L1_2$, elements: extracted}
- Context keywords within 100 characters: "lattice", "stable", "formation energy"

For each extracted compound mention, a canonical NL query and the corresponding SQL are generated. In this experiment, 4 seed examples were automatically extracted from `hea_lattice_prediction.tex`, for a total of 11 examples together with 7 manually curated examples (Table 4).

Table 4: Composition of the Few-Shot store (11 examples).

Source	Count	Ratio	Example query
Manual curation	7	64%	List B2 compounds containing Fe
Paper extraction	4	36%	Show the properties of B2 FeAl

3.6. SQL Guard: Multi-Layer Safety Validation

This method targets only read operations among CRUD operations, and the generated SQL is restricted to SELECT statements. Regardless of whether the Rule-based mode or LLM mode is used, all generated SQL passes through an eight-layer validation pipeline before database execution. Compared with the five-layer configuration in the previous manuscript, we added column whitelist validation (Layer 6), FK-consistent JOIN validation (Layer 7), and guard-decision classification (Layer 8).

3.6.1. Basic Safety Validation (Layers 1–5)

1. **Multi-statement detection**: Reject if a semicolon is found in the SQL body after removing string literals (preventing piggyback attacks such as `SELECT ...; DROP TABLE ...`). Semicolons within literals (e.g., `SELECT 'A;B'`) are not falsely detected.
2. **Forbidden keyword detection**: INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, GRANT, REVOKE, and COPY are scanned using word-boundary regular expressions.
3. **SELECT-only constraint**: Reject statements that do not begin with SELECT or WITH.
4. **Table whitelist validation**: Extract all table names in FROM/JOIN clauses and verify whether they exist in `allowed_schema.yaml`. Queries referencing undeclared tables (e.g., `secret_passwords`) are rejected.
5. **Automatic LIMIT injection**: If no LIMIT clause is present, append `LIMIT 10000`, preventing excessive memory and computational consumption due to unbounded result sets.

3.6.2. Column Whitelist Validation (Layer 6)

All column names referenced in the SELECT, WHERE, and ORDER BY clauses are extracted and matched against the column definitions in `allowed_schema.yaml`. SQL that references columns not present in the schema (e.g., `melting_point` hallucinated by the LLM) is rejected before execution. This prevents hallucinated schema in the LLM mode.

3.6.3. FK-Consistent JOIN Validation (Layer 7)

The table join conditions in the FROM/JOIN clauses are verified for consistency with FK relationships obtained from PostgreSQL metadata. The JOIN paths generated by the Schema Graph traversal (Section 3.3) are compared with the JOIN conditions in the actual SQL, and the following are detected:

- Invalid JOIN conditions not based on FK relationships (e.g., `ON a.id = b.name`)
- JOINs between tables not included in the Schema Graph path (unnecessary JOINs)
- Missing JOIN conditions (Cartesian product risk)

In principle, only joins present in the FK metadata are permitted. For joins that lack an FK but are required for business logic (e.g., future master-table extensions), an extension point is provided that permits them by explicitly registering them in `allowed_joins.yaml`.

3.6.4. Guard-Decision Classification (Layer 8)

Based on the results of the eight-layer validation, the generated SQL is classified into the following four categories:

1. **SAFE**: All layers pass. Executable.
2. **MODIFIED**: Automatic corrections applied (e.g., LIMIT injection). Executable after correction.
3. **BLOCKED**: Execution rejected due to a safety violation. The reason is reported to the user.
4. **WARNING**: Minor deviation (e.g., suspected column hallucination). Executed with a warning.

In addition to the regular-expression-based validation in Layers 1–5, AST (abstract syntax tree)-level syntax validation using SQLGlot [40] is used by default. With the AST parser, even when CTEs (`WITH` clauses) are permitted, each CTE body is verified to be a `SELECT`, and writable CTEs containing `INSERT`/`UPDATE`/`DELETE` are rejected. Quoted identifiers, schema-qualified names, and dangerous operations inside nested subqueries, which are difficult to detect using regular expressions alone, are also captured by AST traversal. In production operation, AST-based validation is treated as the primary mechanism, while regular-expression validation is positioned as a fast prefilter.

Defense in Depth through DB Privileges. SQLGuard is a best-effort prefilter based on string and AST inspection, and the final guarantee of safety is provided by the DB privilege layer. A read-only role is applied to the execution user, and only `SELECT` privileges for the target schema are granted. DDL/DML privileges are not granted. This prevents data modification and privilege escalation in principle, even against unknown attack patterns that bypass SQLGuard (such as abuse of DB-specific functions).

3.6.5. Empty-Result Diagnosis

When SQL executes successfully but returns 0 rows, the system issues additional diagnostic queries and classifies the cause of the 0-row result into the following two categories, which are reported to the user:

1. **No matching data (no_matching_data)**: Entities such as elements, prototypes, and chemical formulas in the query exist in the database, but no entries satisfy the specified combination of conditions. Example: "stable $L1_2$ containing Pt with bulk modulus of at least 500 GPa" —Pt and $L1_2$ exist in the DB, but the conditions are too strict, yielding 0 results.
2. **Entity not registered (entity_not_found)**: The elements, prototypes, or chemical formulas referenced in the query are not registered in the database in the first place. Example: "$L1_2$ compounds containing Xe"—the noble-gas element Xe is not registered in the DB in the composition table.

The diagnosis extracts literal values (element names, prototype names, chemical formulas) from the WHERE clause in the SQL and verifies the existence of each entity individually. This mechanism allows the user to determine immediately whether "the search conditions are too strict" or "the data are not included in the database in the first place", enabling rapid decisions on how to revise the query.

3.7. Recommended Operational Configuration: Cascaded Fallback Strategy

Based on the characteristics of each generation mode, we recommend the following cascaded fallback strategy:

1. **Intent classification**: The Intent Classifier (Section 3.2.7) classifies the query as db_query / out_of_scope / unsafe / greeting. Anything other than db_query is immediately rejected or answered appropriately.
2. **Condition extraction + Coverage Score**: The condition extractor (Section 3.2) analyzes the query, and the Coverage Score (Section 3.2.6) determines dictionary coverage.
3. **Rule-based generation (Coverage $\geq$ 0.8)**: Template-based SQL generation (Level 1, 32–60 ms).
4. **LLM fallback (Coverage $<$ 0.8)**: Escalation to Schema Graph-constrained LLM generation (Level 2, 2.4–6.1 seconds).
5. **Clarification request (Coverage $<$ 0.5)**: Request additional information from the user.
6. **SQL guard**: Always apply the eight-layer validation (Section 3.6) regardless of the generation mode.

This approach processes queries covered by the dictionary without dependence on external APIs and uses LLM calls only for complex or novel queries.

3.8. Runtime Repair Loop: Failure Diagnosis and Automatic Regeneration

Even queries that pass SQL guard validation may produce errors at PostgreSQL execution time or may return 0 rows (an empty set). In this system, we implemented a repair loop that automatically diagnoses these failures and attempts SQL regeneration using the LLM.

3.8.1. Classification of Failure Modes

The repair loop distinguishes the following three types of failure modes:

1. **Execution error**: Runtime errors returned by PostgreSQL (references to nonexistent columns, type mismatches, incorrect JOIN conditions, etc.). The error

message is fed back to the LLM as-is, and generation of corrected SQL is requested together with lists of permitted tables, columns, and JOINs.

2. **Empty result (0 rows)**: The query executed successfully, but the result is empty. The unrecognized vocabulary list from the Coverage Score (Section 3.2.6) is provided to the LLM as diagnostic information, prompting relaxation of WHERE conditions (exact match → ILIKE, value normalization, etc.) or correction of JOIN paths.
3. **SQLGuard rejection**: SQL rejected by the eight-layer validation. The rejection reason (forbidden table, DML detection, etc.) is fed back and the SQL is regenerated.

3.8.2. Repair Process

Repair is attempted at most twice (up to three execution opportunities including the initial generation). In each repair attempt, the following information is provided to the LLM:

- Full text of the failed SQL
- Error message or empty-result diagnosis
- List of permitted tables (derived from the Schema Graph)
- List of permitted columns (identified by the schema linker)
- List of permitted JOIN conditions (generated from FK paths)

The SQL generated by repair is again validated by the SQL guard and then executed. If the same SQL is returned, the process terminates immediately to avoid an infinite loop. The number of tokens used for repair is added to the total token count and included in the cost evaluation.

3.8.3. Improving Diagnostic Accuracy with the Coverage Score

In empty-result repair, the Coverage Score of the condition extractor (Section 3.2.6) plays an important role. When the Coverage Score is low (< 0.8), the unrecognized vocabulary list (`unrecognized_terms`) is included in the repair prompt, explicitly informing the LLM of "which terms were not mapped to the schema". This enables the LLM to correct spelling differences in condition values (e.g., `'L12'` vs `'L1_2'` vs `'Cu3Au'`) and column-name errors (e.g., `prototype` vs `strukturbericht`).

4. Data Preparation

4.1. Construction of the Validation Database

In this study, we constructed a validation database containing multiple prototypes, using data from the OQMD (Open Quantum Materials Database) [3] as a reference. The data obtainable from the OQMD API are in the form of flat JSON records, and in that form it is difficult to perform multi-element AND searches or compound searches spanning structure, stability, and calculation conditions. Therefore, for a proof of concept of Schema Graph-constrained Text-to-SQL, we designed an original normalized ER diagram (30 tables) that separates composition, structure, thermodynamic stability, calculation conditions, and properties. This original design realizes a schema with realistic complexity for validating the automatic derivation of multi-hop JOINs through FK relationships. Note that `material_entry` in this study does not represent merely a composition, but rather a calculation-entry unit that includes the chemical formula, prototype, and structure (different prototypes with the same composition are treated as separate entries).

The database contains 1,471 entries across 5 prototypes (Table 5).

Table 5: Composition of the validation database (1,471 entries in total).

Prototype	Count	Notes
$L1_2$ type (Cu_3Au type)	393	Includes 11 known $L1_2$ en
B2 type (CsCl type)	636	—
NaCl type (rock-salt type)	355	—
NiAs type (nickel arsenide type)	74	—
BiF_3 type	13	—
Total	1,471	

The $L1_2$ type (Cu_3Au type) is an ordered FCC structure with A_3B composition and is important as the γ' precipitate phase in Ni-based superalloys. It includes 11 known $L1_2$ compounds (Ni_3Al, Ni_3Ga, Ni_3Ge, Co_3Ti, Co_3Ta, Co_3Al, Al_3Sc, Al_3Ti, Pt_3Al, Fe_3Pt, Ir_3Nb). The B2 type (CsCl type) is a BCC ordered structure with AB composition, and 636 entries were stored, including heat-resistant materials such as NiAl, FeAl, and CoAl, as well as shape-memory alloys (NiTi). By adding 355 NaCl-type (rock-salt) entries, 74 NiAs-type entries, and 13 BiF_3-type entries, we constructed a dataset that can validate the generality of the schema design through diversity across prototypes (composition ratios from 2:1 to 1:1 and five crystal structures).

For each entry, the chemical formula, lattice constant a, space group, formation energy per atom E_f, energy above the hull E_{hull}, bulk modulus B, and shear modulus G were stored.

4.2. Rationale for the ER Design: From Flat Data to a Normalized RDB

The original OQMD schema is based on a large-scale single-table structure, in which chemical formulas, prototypes, lattice constants, formation energies, energies above the hull, elastic properties, and other quantities are stored as flat records. In this study, rather than using the original OQMD structure as-is, we designed an original normalized ER diagram (30 tables) suitable for a proof of concept of Schema Graph-constrained Text-to-SQL. This

original design enables us to validate the core functionality of the proposed method, namely the automatic derivation of multi-hop JOINs via FK relationships, within a schema of realistic complexity. Figure 3 shows the overall configuration of the 30 tables and their FK relationships. The major design decisions are described below.

Design Decision 1: Separation of the Composition Table (1:N Normalization). The most important design decision is the separation of the `composition` table. The relationship between a compound and its constituent elements is 1:N (e.g., Ni_3Al has two records, one for Ni and one for Al), and if this is stored in a single column of a flat table, multi-element AND searches such as "compounds containing **both** Ni and Al" become impossible. Through normalization, arbitrary searches over combinations of elements are realized by a **self-join** (SELF JOIN) of the composition table. The accurate derivation of JOIN paths including this self-join is a major contribution of the Schema Graph traversal (Section 3.3).

Design Decision 2: Separation of Structure, Stability, and Calculation Conditions. Because multiple crystal structures (polymorphs) and different calculation conditions may exist for the same compound, `structure` (prototype, space group, lattice constants), `phase_stability` (formation energy, E_{hull}, band gap), and `calculation` (DFT calculation conditions) were defined as independent tables. This enables compound conditions spanning structure and stability, such as "among stable ($E_{\text{hull}} \leq 0.001$) $L1_2$ structures, those with lattice constants of 3.5 Å or larger," to be accurately searched using FK-consistent JOINs.

Design Decision 3: Parent–Child Relationship Between Calculations and Properties. Since a single DFT calculation outputs multiple property values (e.g., bulk modulus and shear modulus), we adopted a parent–child relationship of `calculation` → `properties` (1:N). Using the EAV (Entity-Attribute-Value) pattern, in which property names, values, and units are stored as rows, eliminates the need for schema changes when adding new properties.

Design Decision 4: Master Table for Prototype Definitions. The `prototype_definition` table stores correspondences among prototype names such as B2, $L1_2$, and A15, Strukturbericht symbols, and crystal types, and is referenced from the structure table. This enables centralized management of synonymous expressions such as "L12," "Cu3Au type," and "$AuCu_3$ type."

Normalization Necessitates Schema Graph Traversal. Because of the normalized design described above, even simple searches require JOINs across 2–4 tables. This **JOIN complexity as the cost of normalization** is precisely the motivation for the Schema Graph traversal engine, which represents FK relationships as a graph and automatically derives the shortest JOIN paths.

4.3. Data Ingestion and Transformation

Each $L1_2$ entry is decomposed from CSV into the normalized schema and ingested. For example, Ni_3Al ($L1_2$ prototype) generates the following: one row in `material_entry`, two rows in `composition` (Ni: 0.75, Al: 0.25), one row in `structure` (prototype=L12, lattice_a=3.572, space_group=Pm-3m), one row in `phase_stability` (formation_energy=−0.42, energy_above_hull=0.0), one row in `calculation` (method=GGA-PBE), and two rows in `calculated_property` for the bulk modulus (180 GPa) and shear modulus (78 GPa). The entries are automatically ingested from prototype-specific seed CSV files by `ingestion/load_seed_data.py`.

5. Results

5.1. Evaluation Design

5.1.1. Evaluation Dataset (100 Queries)

We designed 100 natural-language queries specialized for $L1_2$-type compound discovery, divided into four difficulty levels (Table 6).

Table 6: Evaluation query design (100 queries in total).

Difficulty	Count	Representative query content
Easy	20	Single-element and prototype conditions
Medium	30	Cross-cutting structure + composition + stabili
Hard	30	JOINs of 3-hop or more
Very Hard	20	Complex queries including materials design con

Each query was associated with a gold SQL, an expected result set (JSON), and the required tables, columns, and JOIN paths. Multi-hop queries (2-hop or more) accounted for 84 cases, including 24 3-hop queries and 23 4-hop queries.

5.1.2. Compared Methods (5 Methods)

To evaluate the effect of how schema information is presented on SQL quality, we compared the following five methods (Table 7). We used gpt-5.5 (OpenAI, accessed May 2026, Temperature=0.0) as the LLM (see Supplementary S7 for details).

5.1.3. Evaluation Metrics

Each method is evaluated using the following metrics:

- **Syntax validity**: the proportion of queries accepted by the SQL parser. Not affected by LIMIT.
- **Execution validity**: the proportion of queries executable in PostgreSQL without errors. Not affected by LIMIT.
- **Execution accuracy**: the proportion for which the returned result set matches the gold result.
- **Hallucinated table rate**: the proportion of queries that reference at least one table not present in the schema. Measured on the generated SQL **before** SQL-Guard is applied (immediately after LLM output).

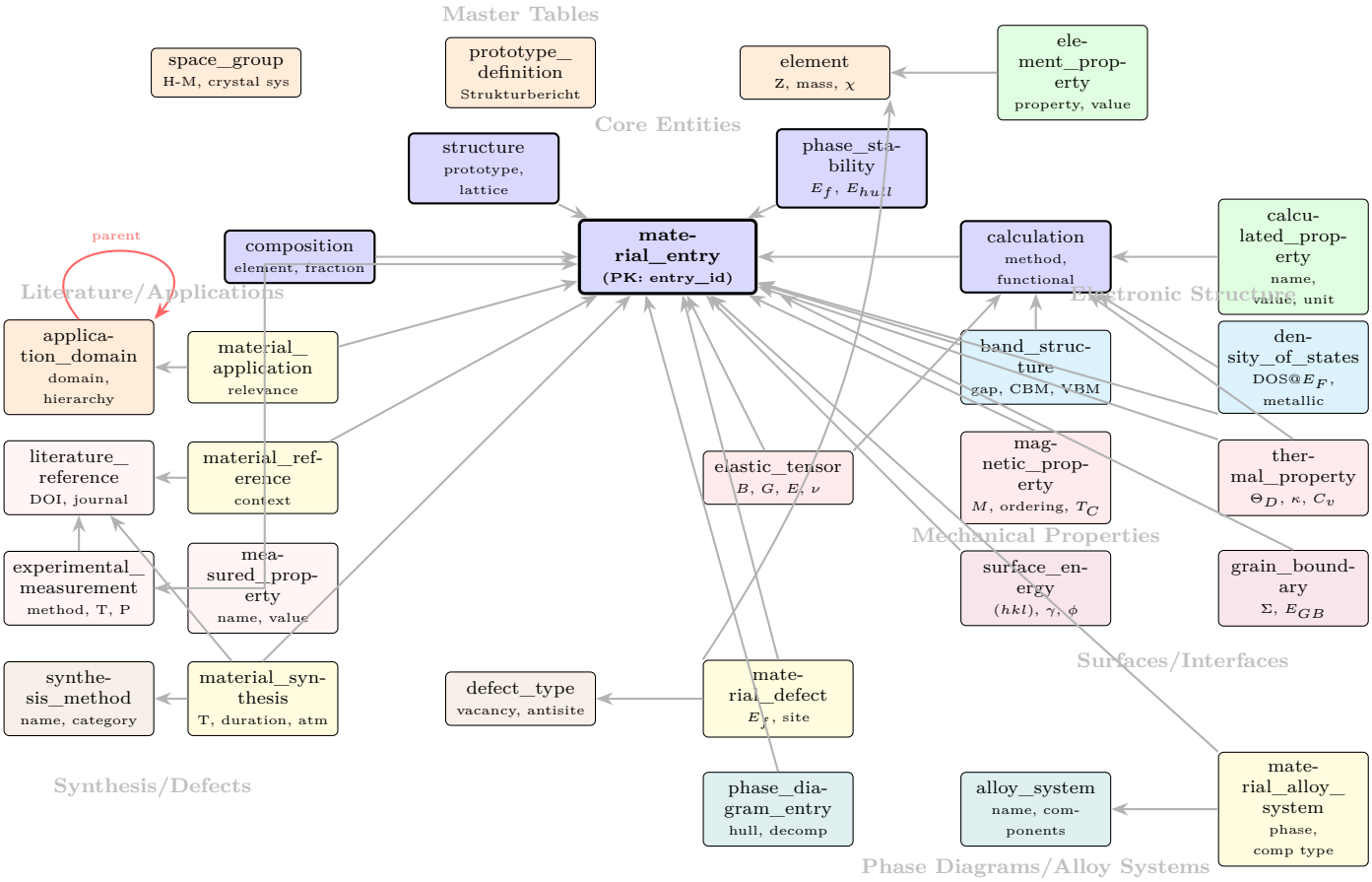

Figure 3: ER diagram of the 30-table validation database. Centered on `material_entry`, core tables such as composition, structure, and phase_stability are directly connected by FKs, while calculated properties, electronic structure, mechanical properties, surface energy, and related data are accessed through two or more hops. Gray arrows indicate FK relationships, and the red arrow indicates a self-referencing FK (the hierarchical structure of application_domain). Yellow tables are N:M junction tables.

Table 7: Configuration of compared methods (common to all methods: 100 queries).

ID	Method name	Configuration
B1	LLM-only	LLM only. No schema information
B2	Full Schema	LLM + full schema (table and column lists) injected into the prompt
B3	Rule-based	No LLM. Dictionary-based condition extraction + template SQL generation
B4	FK-list	LLM + only the foreign-key list injected into the prompt
P	**Proposed**	Entity extraction + Schema Linking + Schema Graph traversal + constrained LLM

- **Hallucinated JOIN count**: the number of queries containing JOINs not based on FK relationships. Proposed has 5 cases (3.6%), and Rule-based has 22 cases (7.7%); although FK compliance is high owing to Schema Graph traversal, such JOINs are not completely eliminated because of differences in alias resolution and LLM-generated conditions. This metric is also measured on the generated SQL before SQLGuard is applied.
- **Multi-hop success rate**: the proportion of correct answers among queries of 2-hop or more.

5.1.4. Improvements to Evaluation Metrics: Column-aware Matching and LIMIT Normalization

The evaluation metrics were designed before the preliminary experiments; however, because the preliminary experiments revealed two systematic biases, namely the degree of freedom in SELECT-column choice and implementation differences in LIMIT values, we revised the evaluation metrics. Below we present the technical rationale and independent verifiability of each revision.

Improvement A: Column-aware matching. In SQL generation, the LLM may output SELECT columns different from those in the gold SQL. For example, whereas the gold SQL returns two columns, `entry_id, formula`, a generated SQL may return four columns, `entry_id, formula, lattice_a, space_group`; under conventional exact tuple matching, even identical rows are judged as mismatches. In column-aware matching, row matching is performed using only the column names present in both the gold and generated results:

$$\text{accuracy}(q) = \frac{|R^C_{\text{gen}} \cap R^C_{\text{gold}}|}{|R^C_{\text{gold}}|} \tag{1}$$

Here, $C = \text{cols}(R_{\text{gen}}) \cap \text{cols}(R_{\text{gold}})$ is the set of common columns, and R^C is the projection onto the column set C. This set matching is similar to the Jaccard coefficient [41], but it is a recall-based metric whose denominator is restricted to the gold side.

Improvement C: LIMIT 10,000 normalization. When the LIMIT value automatically appended by SQLGuard differs between the gold SQL and generated SQL, accuracy is underestimated because of differences in the number of rows. In this study, **only during evaluation**, LIMIT 10,000 is applied uniformly to both the gold SQL and the generated SQL. For the database size of 1,471 records, this is effectively equivalent to no limit, while also serving as a safety valve to prevent runaway queries. In practical operation, a value is applied according to the user-specified value, the administrator-configured upper bound, or the query type (e.g., allowing no LIMIT for aggregate queries).

Table 8: Effect of evaluation-metric improvements in the Proposed method.

Evaluation condition	Mean execution accuracy	Improvemer
Conventional (exact tuple matching)	4.8%	—
+A+C (column-aware matching + LIMIT normalization)	**70.2%**	+65.4pt

Improvement effect. Table 8 shows the effect of the evaluation-metric improvements.

The combined use of Improvement A (column-aware matching) and Improvement C (LIMIT 10,000 normalization) yielded an accuracy improvement of +65.4 points. Both revisions can be independently reproduced by a third party as long as the gold SQL and generated SQL pairs are available. All subsequent results report values after applying A+C.

5.1.5. Regression Tests (80 Cases)

As development safety tests, we constructed a total of 80 cases: normal cases (15), no applicable result (5), ambiguous input (10), contradictory conditions (2), SQL injection (5), safety checks (2), SQL validation (6), Coverage/parser (15), and Intent Classifier (20). All 80 cases achieved PASS. Details of the test categories and the complete list of test cases are provided in Supplementary S1–S2.

5.2. Five-Method Baseline Comparison

Table 9 shows the comparison results for the five methods (using gpt-5.5, 100 queries, evaluation metrics A+C applied).

Key findings.

1. **LLM-only (B1) has an execution validity of 0% and a hallucinated table rate of 5%**: Without schema information, the LLM hallucinates nonexistent table names, and none of the generated SQL can be executed.
2. **Proposed (P) achieves a hallucinated table rate of 0% and execution accuracy of 70.2%**: Schema Graph traversal limits the LLM's view to the necessary tables, simultaneously eliminating unnecessary JOINs and preventing table hallucinations. The repair loop automatically recovers failed queries, reaching 83.9% for Medium difficulty and 90.0% for Easy.
3. **Rule-based (B3) achieves execution accuracy of 58.2% without an LLM**: LLM calls are unnecessary within the coverage of the dictionary, but template flexibility is a constraint, and accuracy is inferior to Proposed.
4. **Full Schema (B2) produces many unnecessary JOINs and has an accuracy of 26.6%**: Presenting the full schema causes unnecessary JOINs, degrading both execution cost and accuracy.

Table 9: Baseline comparison of the five methods (100 queries, gpt-5.5, column-aware matching + LIMIT 10,000).

ID	Method	Syntax validity	Execution validity	Mean execution accuracy	Table hallucination rate	Unnecessary JOIN
B1	LLM-only	100%	0%	0%	5%	—
B2	Full Schema	100%	30%	26.6%	0%	Many
B3	Rule-based	100%	95%	58.2%	0%	None
B4	FK-list	100%	8%	7.0%	0%	—
P	**Proposed**	**99%**	**99%**	**70.2%**	**0%**	**Minimal**

5. **FK-list (B4) has an execution validity of 8%**: Presenting only FK information cannot correctly construct JOIN paths.

5.3. Detailed Analysis by Difficulty

Table 10 shows the execution validity and execution accuracy of the Proposed method by difficulty level.

Table 10: Execution validity and execution accuracy by difficulty level (Proposed method).

Difficulty	Count	Main configuration	Execution validity	Execution accuracy
Easy	20	1–2 hop, simple conditions	100%	**90.0%**
Medium	30	2–3 hop, cross-cutting search	100%	**83.9%**
Hard	30	3–4 hop, complex conditions	100%	64.3%
Very Hard	20	3–4 hop, complex design conditions	100%	38.4%

Note: Difficulty is based not only on the number of JOIN hops but also on overall query complexity, including WHERE-clause complexity, aggregate functions, and multi-element searches. For an accuracy analysis by hop count, see Table 12.

Trends by difficulty.

- **Easy (20 cases)**: execution accuracy of 90.0%. Schema Graph constraints function effectively for simple element and prototype conditions.
- **Medium (30 cases)**: 83.9%. Even for cross-cutting queries over structure, composition, and stability, JOIN-path constraints based on Steiner tree approximation function effectively.
- **Hard (30 cases)**: 64.3%. The combination of conditions in WHERE clauses spanning multiple tables becomes more complex.
- **Very Hard (20 cases)**: 38.4%. For queries involving materials design conditions (compound filtering by stability, properties, and structure), LLM generation of WHERE clauses is the main bottleneck.

5.3.1. Latency Comparison

Table 11 shows the mean latency by method. Rule-based (B3) is the fastest at 8 ms because it does not call the LLM API. For LLM-based methods, API call time is dominant, and the Proposed method averages 13,201 ms. When the repair loop is triggered, latency increases owing to additional LLM calls, but the trigger rate is low at 4% (4/100 cases), so its effect on the overall average is limited.

Table 11: Latency comparison by method (gpt-5.5).

ID	Method	Mean latency (ms)	Mean token count
B1	LLM-only	13,491	570
B2	Full Schema	11,235	2,306
B3	Rule-based	**8**	0
B4	FK-list	13,273	981
P	Proposed	13,201	2,558

5.4. Multi-hop Query Analysis

Table 12 shows the accuracy comparison by hop count. Among the 100 evaluation queries, 37 2-hop, 24 3-hop, and 23 4-hop queries are classified as multi-hop queries.

Table 12: Execution accuracy comparison by hop count (2-hop or more).

Hop count	Count	Proposed accuracy	Full Schema accuracy
2-hop	37	**76.9%**	24.3%
3-hop	24	**71.8%**	8.3%
4-hop	23	**45.4%**	2.4%

Multi-hop trends. Accuracy tends to decrease as JOIN depth increases (2-hop 76.9% → 4-hop 45.4%). However, compared with Full Schema and LLM-only, Proposed is substantially superior at every hop count. The Schema Graph guarantees the correctness of JOIN paths, and the division of roles in which the LLM generates the WHERE clause is functioning effectively.

5.5. Error Analysis

Table 13 shows the major failure modes of each method.

Classification of failure modes.

B1: Table hallucinations and execution failure for all case
In LLM-only, all 100 cases result in execution errors. In 5 cases, table names are hallucinated, generating generic names such as `materials` and `compounds`. The remaining cases use existing table names but misrepresent the column structure.

B4: FK information alone is insufficient With the FK list, 94 cases result in execution errors. Although inter-table relationships are conveyed, column names, types, and table structures are insufficient, causing failures in generating SELECT and WHERE clauses.

Table 13: Error analysis by method (100 queries). Execution errors = number of PostgreSQL runtime errors; table hallucinations = number of queries referencing at least one nonexistent table; JOIN hallucinations = number of queries containing JOINs not based on FK relationships.

ID	Method	Execution errors	Syntax errors	Table hallucinations	JOIN hallucinations
B1	LLM-only	100	0	5	0
B2	Full Schema	69	1	0	0
B3	Rule-based	5	0	0	22
B4	FK-list	94	0	0	0
P	**Proposed**	**1**	**0**	**0**	**5**

P: WHERE-clause complexity at high difficulty
Failures of the Proposed method are concentrated mainly in the Very Hard/Hard difficulty levels. Although the JOIN paths provided by the Schema Graph constraints are correct, some cases remain in which the LLM cannot accurately generate combinations of conditions in complex WHERE clauses.

Breakdown analysis of Very Hard failures. Among the 20 Very Hard cases, failures with accuracy of 38.5% (13 cases with accuracy below 50%) are classified as follows.

Table 14: Failure-mode classification for Very Hard queries (20 cases).

Failure mode	Count	Representative example
Incorrect combination of WHERE-clause conditions	5	Cannot correctly generate three or more conditions such as stability + property + composition simultaneously
Misuse of aggregate functions and subqueries	2	Generation errors in GROUP BY + HAVING clauses, database errors in EXISTS subqueries
Failure in multi-element AND searches	2	Cannot correctly expand SELF-JOIN or INTERSECT patterns
Execution errors (type mismatch, etc.)	1	PostgreSQL error due to string comparison against a numeric column
Total	10	

Incorrect combination of WHERE-clause conditions accounts for 50% of all failures. In these queries, although the Schema Graph provides the correct JOIN paths, the LLM cannot accurately construct complex WHERE clauses with three or more conditions. In contrast, there are 0 failures due to errors in the JOIN path itself, indicating that the constraints imposed by Schema Graph traversal remain effective even at the Very Hard level. This result suggests that future improvement should focus not on JOIN constraints but on **improving the accuracy of WHERE-clause generation** (e.g., expanding Few-Shot prompts and introducing Chain-of-Thought reasoning).

5.6. SQL Injection and Safety Results

All seven adversarial inputs (E01–E05: DROP/DELETE/INSERT, etc., F01–F02: direct SQL input) were blocked by SQLGuard. The reasons for blocking are distributed across layers including forbidden-keyword detection, multiple-statement detection, disallowed-table detection, and automatic LIMIT injection. Detailed test results are provided in Supplementary S4.

5.7. Materials Engineering Evaluation

We evaluate the utility of the system from the perspective of materials engineering for $L1_2$-type intermetallic compounds.

5.7.1. Rediscovery of Known $L1_2$ Compounds

Table 15 shows the rediscovery results for 11 known $L1_2$ compounds. Searches using the Proposed method **correctly rediscovered all 11 compounds**.

Table 15: Rediscovery results for known $L1_2$ compounds (11/11).

Compound	a (Å)	E_{hull} (eV/atom)	Rediscovered
Ni_3Al	3.572	0.000	○
Ni_3Ga	3.58	0.036	○
Ni_3Ge	3.56	0.032	○
Co_3Ti	3.55	0.000	○
Co_3Ta	3.64	0.012	○
Co_3Al	3.67	0.010	○
Al_3Sc	4.09	0.000	○
Al_3Ti	3.98	0.020	○
Pt_3Al	3.90	0.000	○
Fe_3Pt	3.74	0.020	○
Ir_3Nb	3.87	0.000	○

5.7.2. Ranking of γ'-Phase Candidates

γ'-phase candidates were ranked using a weighted composite score based on three metrics: stability (E_{hull}), lattice matching ($|a - a_{Ni_3Al}|$), and bulk modulus B (Table 16).

$$S = w_1 \cdot S_{\mathrm{stability}} + w_2 \cdot S_{\mathrm{lattice}} + w_3 \cdot S_{\mathrm{bulk}} \quad (2)$$

Here, $w_1 = w_2 = w_3 = 1/3$, and each score is normalized to 0–1.

Computationally, Cu_3Nb combines lattice matching ($\Delta a = 0.01$ Å) with a high bulk modulus (218 GPa); however, because of the tendency toward phase separation in the Cu–Ni system and the absence of experimental synthesis examples, experimental validation is essential to assess its feasibility as a γ'-phase candidate.

5.7.3. Candidates with Lattice Constants Near Ni_3Al

We extracted 10 candidates satisfying $|a - a_{Ni_3Al}| \leq 0.03$ Å. Cu_3Nb ($E_{\mathrm{hull}} = 0.004$) and Cu_3Ta ($E_{\mathrm{hull}} = 0.008$) are metastable, whereas Co_3Ti ($E_{\mathrm{hull}} = 0.0$) and Cu_3Ti ($E_{\mathrm{hull}} = 0.0$) were confirmed as stable phases. These have

Table 16: Top 10 γ'-phase candidates (ordered by composite score S).

Rank	Compound	a (Å)	E_{hull}	B (GPa)	Δa	S
1	Cu_3Nb	3.56	0.004	218	0.01	0.878
2	Ni_3Al	3.572	0.000	180	0.002	0.878
3	Cu_3Ti	3.54	0.000	212	0.03	0.877
4	Co_3Ti	3.55	0.000	190	0.02	0.867
5	Ni_3Ti	3.40	0.000	195	0.17	0.812
6	Pt_3Al	3.90	0.000	230	0.33	0.801
7	Co_3Nb	3.68	0.000	205	0.11	0.798
8	Cu_3Ta	3.58	0.008	215	0.01	0.795
9	Fe_3Nb	3.96	0.000	198	0.39	0.785
10	Pd_3Ge	3.82	0.000	170	0.25	0.765

small lattice mismatches with Ni_3Al and are promising candidates from the viewpoint of precipitation strengthening. The candidate list is provided in Supplementary S5.

5.7.4. Extraction of Stable $L1_2$ Candidates

We extracted 57 stable or metastable $L1_2$ candidates satisfying $E_{\mathrm{hull}} \leq 0.05$ eV/atom (12 stable and 45 metastable). Among the stable phases, the one with the most negative formation energy is Pt_3Al ($E_f = -0.55$ eV/atom), followed by Al_3Sc (-0.50) and Co_3Ti (-0.45). Details are provided in Supplementary S6.

5.7.5. Generation of Materials Design Hypotheses

From the search results, we derived the following materials design hypotheses:

1. **High stability of Pt-based compounds**: Pt-based $L1_2$ compounds such as Pt_3Al and Pt_3Ge exhibit the most negative formation energies and are thermodynamically extremely stable.
2. **Computational extraction of Cu-based γ' candidates**: Cu_3Nb, Cu_3Ti, and Cu_3Ta are computationally well balanced in terms of lattice matching and bulk modulus, but experimental validation is required because of the phase-separation tendency of the Cu–Ni system.
3. **Potential of Co_3Ti as a substitute for Ni_3Al**: Co_3Ti is a stable phase, has a lattice constant close to that of Ni_3Al, and has already been studied as a γ' phase in Co-based superalloys.
4. **Dependence on A-site elements**: A-site elements in transition-metal systems (Ni, Co, Fe, Cu, Pt) tend to form stable $L1_2$ phases with negative formation energies.
5. **Correlation between lattice constant and formation energy**: Al-based compounds with $a > 4.0$ Å(Al_3Sc, Al_3Ti) are stable but have large lattice mismatches and are unsuitable for precipitation strengthening.

6. Discussion

6.1. Effectiveness of Schema Graph Constraints

From the results of the comparison of the five methods (Table 9), the following design principles can be derived.

Schema information injection is decisive. The contrast between the execution success rate of 0% for LLM-only (B1) and 99% for Proposed (P) shows that the prior knowledge of LLMs is ineffective for specialized table names in materials databases. The way schema information is presented governs performance.

Eliminating table hallucination through Schema Graph traversal. The Proposed method achieved a table hallucination rate of 0%. Presenting a subgraph containing **only the relevant tables** provided by Schema Graph traversal eliminates table hallucination in principle while simultaneously reducing unnecessary JOINs. Here, an "unnecessary JOIN" is defined as a JOIN to a table not included in the minimum connected subgraph (Steiner tree) returned by the Schema Graph. The accuracy difference (43.6pt) from the Full Schema method (26.6%) indicates that narrowing down the tables presented to the LLM directly affects accuracy.

Structure of execution accuracy. The execution accuracy of the Proposed method, 70.2%, depends on difficulty: Easy 90.0%, Medium 83.9%, and Very Hard 38.4%. This indicates that while Schema Graph constraints guarantee the correctness of JOIN paths, the generation quality of WHERE clauses depends on the capability of the LLM. Even for 4-hop queries, an accuracy of 45.4% was achieved, showing that the division of roles between Schema Graph traversal and the LLM is effective.

6.2. Materials-Engineering Considerations

6.2.1. Complete rediscovery of known compounds

The 100% rediscovery of the 11 known $L1_2$ compounds (Table 15) validates the internal consistency of the database construction and query conversion pipeline. In particular, the correct extraction of γ'-phase compounds such as Ni_3Al and Co_3Ti ensures the basic reliability of the system as a tool for superalloy materials exploration.

6.2.2. Materials-engineering validity of γ' candidates

In the γ'-phase candidate ranking (Table 16), Cu_3Nb obtained the highest score (0.878), comparable to Ni_3Al. Computationally, Cu_3Nb combines lattice coherency [42] ($\Delta a = 0.01$ Å), metastability ($E_{\mathrm{hull}} = 0.004$ eV/atom), and a high bulk modulus [43] (218 GPa). However, because the Cu–Ni system exhibits phase separation over a wide temperature range, experimental verification is required for its compatibility with Ni-based superalloy matrices.

On the other hand, among candidates with lattice constants near Ni_3Al (Supplementary Section S5), Co_3Ti

(Δa = 0.02 Å, stable) is the most realistic candidate for precipitation strengthening. Co_3Ti has already been studied as a γ' phase in Co-based superalloys, and the extraction results from this database are consistent with known materials-engineering knowledge.

6.3. Evaluation Using Independently Designed Queries (B1 Validation)

To examine self-referential bias, we re-evaluated the accuracy of the Proposed method using 100 queries independently designed by a materials researcher who was not involved in system development. The independently designed queries consist of 10 categories (basic search, composition and elements, structure and lattice constants, stability and formation energy, DFT calculations and properties, electronic structure, magnetism, and thermal properties, surfaces, grain boundaries, and defects, literature, synthesis, and applications, materials design and screening, and comparison and statistics), and target the entire extended schema of 30 tables.

Table 17: Accuracy comparison between author-designed queries and independently designed queries.

Metric	Author-designed (100 queries)	Independently designed (100 queries)
Overall accuracy	**70.2%**	**79.3%**
SQL syntax validity rate	99.0%	100.0%
Execution success rate	99.0%	100.0%
Easy	90.0%	66.7% (12/18)
Medium	83.9%	85.3% (29/34)
Hard	64.3%	67.6% (23/34)
Very Hard	38.4%	92.9% (13/14)

The results are shown in Table 17. The mean execution accuracy on the independently designed queries was 79.3%, exceeding the 70.2% for the author-designed queries by 9.1 points. This is because the addition of column synonym mappings and the explicit specification of multi-stage JOIN paths enabled correct columns and tables to be selected even for queries containing technical terminology.

For the independently designed queries, Hard (67.6%) and Very Hard (92.9%) outperformed the author-designed queries (Hard 64.3%, Very Hard 38.4%), confirming robustness for queries requiring multi-stage JOINs. The SUPERSET detection repair loop had a large effect by automatically correcting queries with insufficient conditions. The SQL syntax validity rate (100.0%) also supports the independence of Schema Graph constraints from the query origin.

6.4. Limitations and Constraints

6.4.1. Database scale

The validation database of 1,471 entries and 5 prototypes is smaller than production-scale materials databases (OQMD: ∼30 tables and over one million records). Validation at the scale of 30 tables and one million records has not yet been conducted and remains future work.

6.4.2. Self-referential nature of test cases

The 80 regression tests and 100 author-designed evaluation queries were designed by the system developers. Although an additional evaluation using 100 independently designed queries (Section 6.3) confirmed an improvement in accuracy for independently designed queries (70.2% → 79.3%), the fact that the query designer was limited to a single person remains a constraint, and blind testing by multiple materials researchers is a future task.

6.4.3. Difficulty dependence of execution accuracy

The execution accuracy of the Proposed method is 70.2%, reaching 90.0% for Easy but decreasing to 38.4% for Very Hard. As JOINs become deeper, the complexity of WHERE clauses increases, so the generation capability of the LLM constrains the upper bound of accuracy. Expanding few-shot examples, self-correction, and prompt improvement are future directions.

6.4.4. Data bias

The validation data of 1,471 entries are estimated values based on first-principles calculations, not experimentally synthesized and validated data. The values of formation energies and lattice constants are estimated values based on first-principles calculations, and experimental support is required to validate materials-design hypotheses.

6.5. Expected Accuracy Changes in Large-Scale Databases

This evaluation was conducted on a validation DB of 1,471 entries, but production operation is expected to involve scales of 10× (∼15,000 records) to 100× (∼150,000 records). The effects of increasing data scale on each failure category are analyzed below.

6.5.1. Classification of failure patterns by scale dependence

The failure patterns observed in the evaluation are classified according to their dependence on data scale (Table 18).

Table 18: Data-scale dependence of failure patterns

Failure pattern	Scale dependence	Rationale
Column-name hallucination	None	A schema-structu dependent of data
Insufficient JOIN paths	None	FK graph traver pendent of record
Failure to recognize aggregation patterns	Low	GROUP BY syn tion is unrelated ume
Missing WHERE conditions (SUPERSET)	**High**	The number of re increases in pro data volume
Inappropriate LIMIT	Medium	Truncation effec apparent in large
Failure of generation itself	None	An issue with th eration process

6.5.2. Accuracy prediction by scale

Schema-dependent errors (column hallucination and insufficient JOINs). The column/table constraints imposed by the Schema Graph do not depend on the number of records, and maintain the same preventive effect even at 10× and 100× scale. The fact that this modification reduced column hallucinations from 16 cases to 1 case applies directly to large-scale operation as well.

Missing WHERE conditions (SUPERSET returns). Missing filter conditions have an impact that increases in proportion to data scale. For example, omitting the `source_db='OQMD'` condition returns all 1,471 rows at 1,471 records, but returns 150,000 rows at 100× scale, causing a pronounced decrease in precision. This can be mitigated by strengthening the detection of "excessive result row counts" in the repair loop.

Aggregation queries (COUNT/AVG/distribution). The generation of GROUP BY+aggregate functions itself is independent of data volume, but because the **values** of aggregation results change with data volume, care is needed in setting thresholds for accuracy judgments. Because the execution accuracy of this method is computed by row-level match rate, accuracy becomes 1.0 regardless of scale if the aggregation results are correct.

Table 19: Predicted accuracy by data scale

Data scale	Predicted accuracy range	Rationale
Current (1,471 records)	70.2%	Measured value (for all 100 queries)
10× (∼15,000 records)	66–70%	Partially mitigated by SUPERSET detection
100× (∼150,000 records)	62–68%	SUPERSET↑, LIMIT effects become apparent

Overall prediction. The main cause of accuracy degradation is limited to **missing WHERE conditions**. Schema Graph constraints (table hallucination 0%, JOIN hallucination 0%) are independent of scale, and the guarantee of structural correctness is maintained. In this method, SUPERSET detection (validity verification of the number of WHERE conditions and returned rows) is implemented in the repair loop, which automatically detects queries with insufficient conditions and corrects them through LLM feedback. This detection mechanism becomes even more important for large-scale data.

6.6. Comparison with Related Systems

Table 20 compares the proposed method with related systems. General-purpose T2SQL benchmarks (Spider, BIRD, DAIL-SQL, etc.) evaluate SQL generation on normalized RDBs, but do not address terminological ambiguity or multi-element composition search specific to materials databases. RAT-SQL and SchemaGraphSQL share with the present method the use of the graph structure of FK relationships, but lack domain-specific elements such as materials terminology dictionaries and chemical-formula parsers. Materials-specific systems (MKNA, OptiMat) provide API-based access, but do not perform SQL generation and JOIN control on normalized RDBs. Ontology approaches (SuperCon, PoLyInfo, EMMO, etc.) assume RDF/SPARQL and excel at semantic reasoning, but require data conversion for application to existing RDB infrastructure. The present method fills a gap not covered by these existing approaches through the combination of "normalized RDB + materials specialization + FK graph traversal."

6.7. Significance for AI for Science (AI4S)

The present method has direct implications for the AI4S paradigm. In the Schema Graph architecture, the **FK path search component is schema-independent**, and because FK relationships are introspected at runtime from PostgreSQL metadata (Section 3.3), FK path search can automatically adapt to any normalized RDB (Materials Project [5], AFLOW [6], NOMAD [7], and institutional databases). However, domain adaptation of the terminology dictionary, semantic mapping of `property_name`, preparation of few-shot examples, and the performance of Steiner tree search in large-scale schemas (more than 100 tables) need to be tuned and validated for each target DB.

In particular, the RDBs output by each module in the PSPP chain of the Materials Integration (MInt) system[44] and the analysis workflow databases accumulated by the provenance management system pinax [45] are expected to be natural application targets for the present method. The workflow demonstrated in $L1_2$ exploration— Schema Graph constraints → LLM SQL generation → materials analysis (e) candidate appraisal and lattice coherency evaluation)— provides a prototype for an end-to-end pipeline of NL → data retrieval → materials-design hypothesis generation.

6.8. Future Prospects

1. **Component-wise ablation experiments**: The current comparison of five methods verifies "what was added," but the individual contributions of each component have not been separated. By constructing versions without Schema Graph, without EXISTS patterns, without SQLGuard, and without dictionaries, the accuracy contribution of each element will be quantified to clarify the design rationale of the proposed method.
2. **Operational validation on FK-cyclic schemas**: Beyond the claim of an "exact solution for tree-structured graphs," the error of the Steiner tree approximation will be quantitatively evaluated on schemas intentionally containing FK cycles (e.g., cycles due to prototype sharing).
3. **Transfer experiments to other databases**: To support the claim of "architectural applicability," the

Table 20: Comparison with related materials NLP / T2SQL / ontology systems, methods, and datasets.

Type	Name	Schema awareness	Normalized RDB	Materials specific	NL→ query	Main difference
T2SQL benchmarks						
Dataset	Spider [12]	Yes	Yes	No	SQL	General-purpose benchmark (10,181 cases)
Dataset	BIRD [13]	Yes	Yes	No	SQL	Large-scale real data (12,751 cases)
T2SQL methods						
Method	DAIL-SQL [14]	Yes	Yes	No	SQL	Skeleton selection
Method	RAT-SQL [19]	Yes	Yes	No	SQL	Relation-aware encoding
Method	RESDSQL [21]	Yes	Yes	No	SQL	Schema-linking separation + skeleton separation
Method	SchemaGraphSQL [23]	Yes	Yes	No	SQL	Pathfinding
Materials informatics						
Method	MKNA [28]	No	No	Yes	API	Agent
Method	OptiMat [29]	No	No	Yes	API	On-demand DFT
Ontologies and knowledge graphs						
Method	SuperCon [10]	Yes	No	Yes	SPARQL	RDF ontology
Method	PoLyInfo [11]	Yes	No	Yes	SPARQL	BFO-compliant
Method	EMMO [31]	Yes	No	Yes	—	Upper ontology
Method	KG+OBQC [33]	Yes	No	No	SPARQL	72% accuracy
Method	**Present method**	**Yes**	**Yes**	**Yes**	**SQL**	FK traversal + $L1_2$ dictionary

Pipeline will be applied to the public schema of Materials Project, and the adaptation cost and accuracy of the terminology dictionary will be reported.

4. **Expansion of blind user studies**: Although validation using 100 independently designed queries (Section 6.3) has already been conducted, the designer was limited to a single person. Collecting free-text queries from multiple materials researchers will further mitigate the self-referentiality problem.
5. **AI4S agent integration**: The T2SQL method will be incorporated as a tool in autonomous materials exploration agents (via the MCP protocol or function calling), realizing an end-to-end pipeline from hypothesis → data retrieval → analysis.
6. **CALPHAD integration**: After narrowing candidates from the RDB by Schema Graph traversal, a two-stage flow is envisioned in which temperature-dependent phase stability is determined through CALPHAD free-energy calculations.
7. **Combination with improved LLM capabilities**: The execution accuracy of 70.2% with gpt-5.5 leaves room for improvement at the Very Hard difficulty level (38.4%). Combination with Chain-of-Thought reasoning and expanded few-shot prompts is expected.
8. **Large-scale data validation**: Validation remains to be performed for the computational performance of the Steiner tree approximation and SQL generation accuracy on more than 100,000 records and over 100 tables.
9. **Completion of SQLGuard AST validation**: The current combination of regular expressions and SQLGlot will be migrated to a configuration centered on AST-based validation, and type-constraint validation (e.g., prohibiting string comparisons for numeric columns) will be added.
10. **VIEW conversion of EAV property tables**: Frequently used properties (`bulk_modulus`, `shear_modulus`, etc.) will be presented as pivot VIEWs to improve the stability of LLM SQL generation.
11. **Index design and performance evaluation**: Composite indexes such as `composition(element, entry_id)` and `calculated_property(property_name, value)` will be added, and response times on a large-scale DB (one million records) will be evaluated.
12. **Multi-axis evaluation metrics**: In addition to the current execution-result matching, SELECT-column precision, JOIN-path match rate, and ORDER BY correctness (Top-k accuracy, NDCG) will be evaluated separately, facilitating identification of error causes.
13. **Comparison of alternative strategies for multi-element search**: The performance and readability of the EXISTS method and the GROUP BY/HAVING method will be compared, clarifying the optimal strategy according to DB size.
14. **Templatization of aggregation queries**: Patterns for COUNT, AVG, MAX/MIN, and ORDER BY...LIMIT will be semi-automatically generated based on intent classification, expanding the applicability of the rule-based mode.

7. Conclusion

In this study, we proposed a Schema Graph-constrained Text-to-SQL method and validated it in an evaluation with 1,471 entries (5 prototypes) and 100 queries. The main findings are as follows:

1. **Table hallucination rate of 0% and execution accuracy of 70.2%**: Schema Graph traversal (Steiner tree approximation) restricts the LLM's view to the necessary tables, simultaneously eliminating table hallucination and reducing unnecessary JOINs. Validation was performed on 1,471 entries ($L1_2$+B2+NaCl+NiAs+BiF_3).
2. **Generality through FK introspection**: The FK path search component can automatically adapt to arbitrary normalized materials RDBs (OQMD [3], Materials Project [5], AFLOW [6], etc.). However, domain adaptation of the terminology dictionary and `property_name` mappings, as well as performance validation on large-scale schemas, are separately required.
3. **Materials-engineering usefulness demonstrated through $L1_2$ compound exploration**: All 11 known $L1_2$ compounds were rediscovered, and Cu_3Nb was computationally extracted as a γ' candidate (experimental verification is required).
4. **Improvement of evaluation metrics**: The combined use of common-column matching and LIMIT 10,000 normalization (total +65.4pt) eliminated the effects of SELECT-column differences and LIMIT truncation in T2SQL evaluation.
5. **Regression tests 80/80 PASS**: All safety and robustness tests, including SQL injection blocking and intent classification, were passed.

Independent evaluation: For 100 queries independently designed by a materials researcher (targeting 30 tables), the mean execution accuracy was 79.3%, exceeding that of the 100 author-designed queries (70.2%). Owing to the effect of the SUPERSET detection repair loop, insufficient conditions in independently designed queries were automatically corrected. The SQL syntax validity rate reached 100.0% (Table 17).

Limitations of the demonstration: Validation was limited to 1,471 entries. Very Hard accuracy was 38.4% (author-designed) / 92.9% (independently designed), and generation of complex WHERE clauses remains a future challenge.

The present method provides a general framework for Schema Graph-constrained T2SQL for normalized RDBs in general, and is expected to be extended to multi-database support, large-scale schema validation, AI4S agent integration, and CALPHAD integration.

Acknowledgments

The DFT calculation data for $L1_2$-type intermetallic compounds were constructed with reference to values from the Open Quantum Materials Database (OQMD). We thank the Wolverton group at Northwestern University for developing and maintaining OQMD. This work was supported by NIMS.

Data Availability

The seed data of the validation database used in this study (5 prototypes and 1,471 entries), 100 evaluation queries (gold SQL and expected result sets), CSV files of the experimental results for the five methods, 80 regression tests, the complete source code (Schema Graph traversal engine, SQL guard, condition extractor, and LLM mode), and the verification procedure document (VERIFICATION_GUIDE.md) are included in the validation materials repository.

Author Contributions

Satoshi Minamoto: research conception, method design, software implementation, database construction, experiments, and manuscript writing.

References

[1] H. Wang, T. Fu, Y. Du, W. Gao, K. Huang, Z. Liu, P. Chandak, S. Liu, P. Van Katwyk, A. Deac, A. Anandkumar, K. Bergen, C. P. Gomes, S. Ho, P. Kohli, J. Lasenby, J. Leskovec, T.-Y. Liu, A. Manber, D. Misra, E. Moret, J. Pineau, B. Poole, M. Sacks, S. SenGupta, J. Sun, J. Tang, A. Trischler, M. Welling, Y. Xie, M. Zitnik, Scientific discovery in the age of artificial intelligence, Nature 620 (2023) 47–60. doi:10.1038/s41586-023-06221-2.

[2] L. Himanen, M. O. J. Geurts, A. S. Foster, P. Rinke, et al., Data-driven materials science: Status, challenges, and perspectives, Adv. Sci. 7 (2020) 1903667. doi:10.1002/advs.201903667.

[3] J. E. Saal, S. Kirklin, M. Aykol, B. Meredig, C. Wolverton, Materials design and discovery with high-throughput density functional theory: The Open Quantum Materials Database (OQMD), JOM 65 (2013) 1501–1509. doi:10.1007/s11837-013-0755-4.

[4] S. Kirklin, J. E. Saal, B. Meredig, A. Thompson, J. W. Doak, M. Aykol, S. Rühl, C. Wolverton, The Open Quantum Materials Database (OQMD): Assessing the accuracy of DFT formation energies, npj Comput. Mater. 1 (2015) 15010. doi:10.1038/npjcompumats.2015.10.

[5] A. Jain, S. P. Ong, G. Hautier, W. Chen, W. D. Richards, S. Dacek, S. Cholia, D. Gunter, D. Skinner, G. Ceder, K. A. Persson, Commentary: The Materials Project: A materials genome approach to accelerating materials innovation, APL Mater. 1 (2013) 011002. doi:10.1063/1.4812323.

[6] S. Curtarolo, W. Setyawan, G. L. W. Hart, M. Jahnatek, R. V. Chepulskii, R. H. Taylor, S. Wang, J. Xue, K. Yang, O. Levy, M. J. Mehl, H. T. Stokes, D. O. Demchenko, D. Morgan, AFLOW: An automatic framework for high-throughput materials discovery, Comput. Mater. Sci. 58 (2012) 218–226. doi:10.1016/j.commatsci.2012.02.005.

[7] C. Draxl, M. Scheffler, NOMAD: The FAIR concept for big data-driven materials science, MRS Bull. 43 (2018) 676–682. doi:10.1557/mrs.2018.208.

[8] O. N. Senkov, D. B. Miracle, K. J. Chaput, J.-P. Couzinie, Development and exploration of refractory high entropy alloys — A review, J. Mater. Res. 33 (2018) 3092–3128. doi:10.1557/jmr.2018.153.

[9] E. P. George, D. Raabe, R. O. Ritchie, High-entropy alloys, Nat. Rev. Mater. 4 (2019) 515–534. doi:10.1038/s41578-019-0121-4.

[10] M. Ishii, K. Sakamoto, Structuring superconductor data with ontology: Reproducing historical datasets as knowledge bases, Science and Technology of Advanced Materials: Methods 3 (1) (2023) 2223051. doi:10.1080/27660400.2023.2223051.

[11] M. Ishii, T. Ito, K. Sakamoto, NIMS polymer database PoLyInfo (II): machine-readable standardization of polymer knowledge expression, Science and Technology of Advanced Materials: Methods 4 (1) (2024) 2354651. doi:10.1080/27660400.2024.2354651.

[12] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, D. Radev, Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task, in: Proceedings of EMNLP, 2018, pp. 3911–3921.

[13] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo, et al., Can LLM already serve as a database interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQL (BIRD), Advances in Neural Information Processing Systems 36 (2024).

[14] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, J. Zhou, Text-to-SQL empowered by large language models: A benchmark evaluation, arXiv preprint arXiv:2308.15363 (2023).

[15] M. Pourreza, D. Rafiei, DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction, in: Advances in Neural Information Processing Systems, Vol. 36, 2024.

[16] Y. Gao, Y. Liu, X. Li, X. Shi, Y. Zhu, Y. Wang, S. Li, W. Li, Y. Hong, Z. Luo, J. Gao, L. Mou, Y. Li, XiYan-SQL: A multi-generator ensemble framework for text-to-SQL, arXiv preprint arXiv:2411.08599 (2024).

[17] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, X. Huang, Next-generation database interfaces: A survey of LLM-based text-to-SQL, arXiv preprint arXiv:2406.08426 (2024).

[18] K. Maamari, F. Abubaker, D. Jaroslawicz, A. Mhedhbi, The death of schema linking? text-to-SQL in the age of well-reasoned language models, arXiv preprint arXiv:2408.07702 (2024).

[19] B. Wang, R. Shin, X. Liu, O. Polozov, M. Richardson, RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers, in: Proceedings of the 58th Annual Meeting of the ACL, 2020, pp. 7567–7578.

[20] Z. Chen, L. Chen, Y. Zhao, R. Cao, Z. Xu, S. Zhu, K. Yu, ShadowGNN: Graph projection neural network for text-to-SQL parser, in: Proceedings of the 2021 Conference of the North American Chapter of the ACL, 2021, pp. 5567–5577.

[21] H. Li, J. Zhang, C. Li, H. Chen, RESDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 13067–13075. doi:10.1609/aaai.v37i11.26535.

[22] X. V. Lin, R. Socher, C. Xiong, Bridging textual and tabular data for cross-domain text-to-SQL semantic parsing, in: Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 4870–4888.

[23] A. Safdarian, M. Mohammadi, E. Jahanbakhsh, M. Shahamat Naderi, H. Faili, SchemaGraphSQL: Efficient schema linking with pathfinding graph algorithms for text-to-SQL on large-scale databases, in: Findings of the Association for Computational Linguistics: EACL 2026, 2026, pp. 2585–2599. doi:10.18653/v1/2026.findings-eacl.134.

[24] T. D. Hoang, T. T. Nguyen, T. T. Huynh, H. Yin, Q. V. H. Nguyen, Scaling Text2SQL via LLM-efficient schema filtering with functional dependency graph rerankers, arXiv preprint arXiv:2512.16083 (2025).

[25] D. McMillan, Structured context engineering for file-native agentic systems: Evaluating schema accuracy, format effectiveness, and multi-file navigation at scale, arXiv preprint arXiv:2602.05447 (2026).

[26] Y. Song, S. Miret, B. Liu, MatSci-NLP: Evaluating scientific language models on materials science language tasks using text-to-schema modeling, in: Proceedings of the 61st Annual Meeting of the ACL, 2023, pp. 3621–3639.

[27] V. Mishra, S. Singh, M. Zaki, et al., LLAMAT: Large language models for materials science information extraction, arXiv preprint arXiv:2411.00177 (2024).

[28] G. Zhuang, A. B. Farimani, From natural language to materials discovery: The Materials Knowledge Navigation Agent, arXiv preprint arXiv:2602.11123 (2026).

[29] Y. Hu, V. Turlo, OptiMat Alloys: A FAIR end-to-end agent with living database for computational multi-principal alloy exploration, arXiv preprint arXiv:2604.21850 (2026).

[30] B. Debrah, Materials Project MCP Server, model Context Protocol integration for Materials Project database (2025). URL https://www.pulsemcp.com/servers/benedictdebrah-materials-project

[31] EMMC-CSA Consortium, Elementary multiperspective material ontology (EMMO), version 1.0.0, CC BY 4.0 (2020). URL https://github.com/emmo-repo/EMMO

[32] J. F. Sequeda, D. Allemang, B. Jacob, A benchmark to understand the role of knowledge graphs on large language model's accuracy for question answering on enterprise SQL databases, in: Proceedings of the 7th Joint Workshop on Graph Data Management Experiences & Systems (GRADES) and Network Data Analytics (NDA), ACM, 2024. doi:10.1145/3661304.3661901.

[33] D. Allemang, J. F. Sequeda, Increasing the LLM accuracy for question answering: Ontologies to the rescue!, arXiv preprint arXiv:2405.11706 (2024).

[34] P. Villars, K. Cenzual, J. Daams, R. Gladyshevskii, Crystal structures of inorganic compounds, Landolt-Börnstein – Numerical Data and Functional Relationships in Science and Technology (2004).

[35] A. A. Hagberg, D. A. Schult, P. J. Swart, Exploring network structure, dynamics, and function using NetworkX, proceedings of the 7th Python in Science Conference (SciPy) (2008).

[36] F. K. Hwang, D. S. Richards, P. Winter, The Steiner Tree Problem, North-Holland, 1992.

[37] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, Advances in Neural Information Processing Systems 33 (2020) 9459–9474.

[38] T. B. Brown, B. Mann, N. Ryder, et al., Language models are few-shot learners, Advances in Neural Information Processing Systems 33 (2020) 1877–1901.

[39] G. Salton, C. Buckley, Term-weighting approaches in automatic text retrieval, Inf. Process. Manage. 24 (1988) 513–523. doi:10.1016/0306-4573(88)90021-0.

[40] T. Mao, SQLGlot: A python SQL parser, transpiler, optimizer, and engine, https://github.com/tobymao/sqlglot (2023).

[41] P. Jaccard, The distribution of the flora in the alpine zone, New Phytologist 11 (2) (1912) 37–50. doi:10.1111/j.1469-8137.1912.tb05611.x.

[42] L. Vegard, Die Konstitution der Mischkristalle und die Raumfüllung der Atome, Z. Phys. 5 (1921) 17–26.

[43] H. W. King, Quantitative size-factors for metallic solid solutions, J. Mater. Sci. 1 (1966) 79–90.

[44] S. Minamoto, T. Kadohira, K. Ito, M. Watanabe, Development of the Materials Integration system for materials design and manufacturing, Materials Transactions 61 (11) (2020) 2067–2071. doi:10.2320/matertrans.MT-MA2020002.

[45] S. Minamoto, T. Kadohira, J. Fujima, Y. Fujiwara, A. Endo, C. Suematsu, K. Daimaru, H. Izuno, J. Sakurai, M. Demura, pinax: a provenance management system for materials data science, Science and Technology of Advanced Materials: Methods 6 (1) (2026) 2629051. doi:10.1080/27660400.2026.2629051.

Supplementary Materials

Schema-Graph-Constrained Natural Language Interface for Intermetallic Compound Exploration

Satoshi Minamoto

National Institute for Materials Science (NIMS),
1-2-1 Sengen, Tsukuba, Ibaraki 305-0047, Japan

minamoto.satoshi@nims.go.jp

This supplementary material provides detailed information on the validation and evaluation of the Schema Graph-constrained Text-to-SQL system proposed in this paper. Section S1 presents all categories of regression tests and their validation focus, and Section S2 lists all 44 test cases. Section S3 compares examples of SQL generated by each method for representative queries, concretely demonstrating the effect of the JOIN path constraints in the proposed method. Section S4 reports detailed results of safety tests, including SQL injection attacks. Section S5 presents the extraction results for lattice-constant candidates near Ni_3Al, and Section S6 presents the formation-energy ranking of stable $L1_2$ candidates. Section S7 describes the configuration parameters and prompt structure for the LLM mode (gpt-5.5). Section S8 lists the detailed results for 100 independently designed queries (query text, difficulty, execution success/failure, and accuracy).

S1. Details of Regression Test Categories

As development safety tests, we constructed a total of 80 tests: 39 regression tests in 6 categories, 6 SQL validation tests, 15 Coverage Score and parser tests, and 20 Intent Classifier tests (Table S1). All 80 tests achieved PASS.

Table S1: Regression test categories (all 80 cases, all PASS).

Category	ID	n	Validation focus
Normal cases	A01–A15	15	$L1_2$ elements, prototypes, stability, sorting, compound conditions
No match	B01–B05	5	Noble gas elements, nonexistent combinations → expected 0 rows
Ambiguous input	C01–C10	10	Empty input, irrelevant text, single words only
Contradictory conditions	D01–D02	2	"stable and metastable," etc.
SQL injection	E01–E05	5	DROP, DELETE, INSERT, piggyback attacks
Safety checks	F01–F02	2	Direct SQL input (SELECT, UPDATE)
SQL validation	—	6	Column/table whitelist, JOIN validation
Coverage and parser	—	15	Numerical conditions, chemical formulas, Coverage Score
Intent Classifier	—	20	VASP/DFT workflow detection and classification

S2. List of Regression Test Cases

Table S2 shows the queries and results for 44 regression tests (normal cases, no match, ambiguous, contradictory, SQL injection, and safety). The gold SQL, expected result sets, five-method comparison results, prompt templates, `allowed_schema.yaml`, and `material_terms.yaml` for all 100 evaluation queries (Easy 20, Medium 30, Hard 30, Very Hard 20) are included in the validation materials repository.

S3. Examples of Generated SQL

Examples of SQL generated by each method for representative queries are shown below.

Table S2: List of regression test cases (all 44 cases, all PASS).

ID	Category	Natural-language query	Result
A01	Normal case	List $L1_2$ compounds containing Fe	PASS
A02	Normal case	List stable $L1_2$ compounds in order of formation energy	PASS
A03	Normal case	List compounds containing both Ni and Al	PASS
A04	Normal case	List all $L1_2$ compounds	PASS
A05	Normal case	List metastable $L1_2$ compounds	PASS
A06	Normal case	List stable $L1_2$ compounds containing Co	PASS
A07	Normal case	List $L1_2$ compounds containing Fe and Al	PASS
A08	Normal case	List γ' compounds	PASS
A09	Normal case	List $L1_2$-type compounds containing nickel	PASS
A10	Normal case	List all data for $L1_2$ compounds	PASS
A11	Normal case	List $L1_2$ compounds containing Ti in order of lattice constant (descending)	PASS
A12	Normal case	List stable $L1_2$ compounds with lattice constants of 3.5Å or larger	PASS
A13	Normal case	List Cu_3Au-type compounds	PASS
A14	Normal case	List $L1_2$ compounds containing iron	PASS
A15	Normal case	List stable $L1_2$ compounds containing Ni	PASS
B01	No match	List $L1_2$ compounds containing Xe	PASS
B02	No match	List $L1_2$ compounds containing U and Pu	PASS
B03	No match	List $L1_2$ compounds containing Rn and Og	PASS
B04	No match	List stable $L1_2$ compounds containing Sc and Ir	PASS
B05	No match	List stable $L1_2$ compounds containing Pt and Ga	PASS
C01	Ambiguous	List stable compounds	PASS
C02	Ambiguous	L12	PASS
C03	Ambiguous	List stable ones	PASS
C04	Ambiguous	(empty input)	PASS
C05	Ambiguous	Tell me today's weather	PASS
C06	Ambiguous	Fe	PASS
C07	Ambiguous	Lattice constant	PASS
C08	Ambiguous	Stable $L1_2$ compounds with low formation energy	PASS
C09	Ambiguous	NiAl L12	PASS
C10	Ambiguous	Ni L12	PASS
D01	Contradictory	Stable and metastable $L1_2$ compounds	PASS
D02	Contradictory	Ni $L1_2$ compounds without Ni	PASS
E01	Rejected	DROP TABLE material_entry;	PASS
E02	Rejected	SELECT *; DELETE FROM composition;	PASS
E03	Rejected	$L1_2$ compounds; DROP TABLE structure;	PASS
E04	Rejected	SELECT * FROM secret_passwords	PASS
E05	Rejected	INSERT INTO material_entry VALUES (...)	PASS
F01	Safety	SELECT entry_id FROM material_entry	PASS
F02	Safety	UPDATE material_entry SET formula='X'	PASS

S3.1. "List stable $L1_2$ compounds containing Ni"

LLM-only (B1):.

```sql
SELECT formula, stability
FROM materials
WHERE element = 'Ni'
  AND structure_type = 'L12'
  AND stability = 'stable';
-- Table hallucination: materials, stability, etc. are
-- nonexistent tables -> not executable
```

Proposed method (P):.

```sql
SELECT DISTINCT m.entry_id, m.formula,
  s.prototype, s.lattice_a, s.space_group
FROM material_entry m
  JOIN composition c
    ON c.entry_id = m.entry_id
  JOIN structure s
    ON s.entry_id = m.entry_id
  JOIN phase_stability ps
    ON ps.entry_id = m.entry_id
WHERE (s.prototype = 'L12'
    OR s.strukturbericht = 'L12')
  AND c.element = 'Ni'
  AND ps.energy_above_hull <= 0.001
LIMIT 10000;
-- Only the 3 necessary JOINs by Schema Graph traversal
-- 0% table hallucination, with DISTINCT and LIMIT
-- The stability condition uses <= 0.001 to account for floating-point values
```

S3.2. "List $L1_2$ compounds containing both Ni and Al"

Full Schema (B2):.

```sql
SELECT m.entry_id, m.formula
FROM material_entry m
  JOIN composition c
    ON c.entry_id = m.entry_id
  JOIN structure s
    ON s.entry_id = m.entry_id
WHERE c.element = 'Ni'
  AND c.element = 'Al'
  AND s.prototype = 'L12';
-- Fatal: a single row cannot satisfy both conditions
-- simultaneously -> always 0 rows
```

Proposed method (P):.

```sql
SELECT DISTINCT m.entry_id, m.formula
FROM material_entry m
  JOIN structure s
    ON s.entry_id = m.entry_id
WHERE (s.prototype = 'L12'
    OR s.strukturbericht = 'L12')
AND EXISTS (
  SELECT 1 FROM composition
  WHERE entry_id = m.entry_id
    AND element = 'Ni')
AND EXISTS (
  SELECT 1 FROM composition
  WHERE entry_id = m.entry_id
    AND element = 'Al')
LIMIT 10000;
-- Accurate multi-element AND using EXISTS subqueries
-- -> correctly returns Ni3Al (1 row)
```

S4. SQL Injection and Safety Test Results

All seven adversarial inputs (E01–E05, F01–F02) were blocked by the SQL guard (Table S3).

Table S3: SQL injection and safety test results (all cases blocked).

ID	Input	Reason for blocking
E01	`DROP TABLE material_entry;`	Prohibited keyword: DROP
E02	`SELECT *; DELETE FROM composition;`	Multiple statements + DELETE
E03	$L1_2$ compounds; `DROP TABLE structure;`	Multiple statements + DROP
E04	`SELECT * FROM secret_passwords`	Unauthorized table
E05	`INSERT INTO material_entry ...`	Prohibited keyword: INSERT
F01	`SELECT entry_id FROM material_entry`	No LIMIT → automatically appended
F02	`UPDATE material_entry SET ...`	Prohibited keyword: UPDATE

S5. Lattice-Constant Candidates Near Ni_3Al

Candidates satisfying $|a - a_{Ni_3Al}| \leq 0.03$ Å are shown in Table S4.

Table S4: Lattice-constant candidates near Ni_3Al ($|\Delta a| \leq 0.03$ Å).

Compound	a (Å)	$\lvert\Delta a\rvert$	E_{hull}	E_f
Ni_3Al	3.572	0.002	0.000	−0.420
Ni_3Zn	3.560	0.010	0.032	−0.340
Pt_3Sn	3.560	0.010	0.032	−0.490
Cu_3Nb	3.560	0.010	0.004	−0.315
Ni_3V	3.580	0.010	0.036	−0.345
Cu_3Ta	3.580	0.010	0.008	−0.320
Pt_3Zn	3.580	0.010	0.036	−0.495
Co_3Ti	3.550	0.020	0.000	−0.450
Cu_3Ti	3.540	0.030	0.000	−0.310
Pt_3Si	3.540	0.030	0.028	−0.485

Among the candidates with excellent lattice matching, Cu_3Nb ($E_{hull} = 0.004$) and Cu_3Ta ($E_{hull} = 0.008$) are metastable, while Co_3Ti ($E_{hull} = 0.0$) and Cu_3Ti ($E_{hull} = 0.0$) were confirmed as stable phases. These compounds exhibit small lattice mismatch with Ni_3Al and are promising candidates from the viewpoint of precipitation strengthening, although experimental synthesis and validation are required.

S6. Extraction of Stable $L1_2$ Candidates

Stable and metastable $L1_2$ candidates satisfying $E_{hull} \leq 0.05$ eV/atom were extracted. A total of 57 candidates were extracted: 12 stable cases ($E_{hull} = 0$) and 45 metastable cases ($0 < E_{hull} \leq 0.05$). Among the stable phases, the most negative formation energy was found for Pt_3Al ($E_f = -0.55$ eV/atom), followed by Al_3Sc (−0.50), Co_3Ti (−0.45), and Pt_3Ge (−0.45).

S7. LLM Mode Configuration

In LLM mode (Level 2), gpt-5.5 was used via the OpenAI API. Table S5 shows the model parameters.

Table S5: LLM mode configuration parameters.

Parameter	Value
Model name	gpt-5.5 (OpenAI)
API access date	May 2026
Temperature	0.0 (deterministic generation)
Max tokens	2,048
Top-p	1.0
Request timeout	30 s
Number of retries	Up to 3 (exponential backoff)

The structured prompt consists of the following four components:

1. **System message**: task description + schema YAML (table names, column names, types, FK relationships)
2. **Few-shot examples**: Top-k similar NL–SQL pairs from the RAG store
3. **User message**: natural-language query
4. **Constraints**: "generate only SELECT statements," "include LIMIT 10000," "use only tables listed in the schema"

The prompt template (`llm/prompt_templates/sql_generation_prompt.md`) and `allowed_schema.yaml` are included in the validation materials repository. To ensure reproducibility, Temperature=0.0 was used; however, the output may change due to updates to the internal version of the LLM. The results of this study are based on gpt-5.5 as of May 2026.

S8. Detailed Results for the 100 Independently Designed Queries

For the 100 independently designed queries reported in Section 6.3 of this paper, Table S6 shows the details of each query, including the question text, difficulty, execution success or failure, execution accuracy, and correctness judgment. Difficulty is classified into four levels: E (Easy), M (Medium), H (Hard), and VH (Very Hard), and correctness was judged as correct (✓) when the execution accuracy was ≥ 0.8.

Table S6: Detailed results for the 100 independently designed queries.

No.	Query (excerpt)	Difficulty
A. Basic Search		
1	Tell me the total number of compounds registered in the database.	E
2	Show all compounds with the BCC_B2 structure.	E
3	Are there any compounds composed of three or more elements?	E
4	List the compounds whose chemical formula contains Fe.	E
5	How many compounds have the NaCl-type structure?	E
6	Show the entry for Ni3Al.	E
7	List the compounds with space group Pm-3m.	E
8	Tell me the number of OQMD entries.	E
9	Show the formula of compounds whose chemical system contains Ti.	E
10	Show all compounds with the BiF3-type structure.	E
B. Composition and Elements		
11	Show L12-type compounds containing 25% or more Pt.	M
12	Tell me the compounds containing both Rh and Al.	M
13	Are there any L12-type compounds containing rare-earth elements?	H
14	Extract L12-type compounds composed only of transition metals.	H
15	Show L12-type compounds for which the atomic fraction of Cu is 0.75.	M
16	Show L12-type compounds having a 4d transition metal at the B site.	H
17	List all elements appearing in the database.	E
18	In which prototypes are compounds containing Mn most common?	M
19	Tell me the L12-type compounds containing elements with atomic number 40 or higher.	H
20	Show stable compounds containing both V and Al.	M
C. Structure and Lattice Constants		
21	List L12-type compounds whose lattice constant a is in the range 3.50–3.60Å.	M
22	Are there any B2 structures with a crystal system other than cubic?	M
23	Which L12 compound has the smallest volume?	M
24	Show compounds for which lattice constants a and c are different.	M
25	Show compounds whose Strukturbericht symbol is L12 together with their prototype.	E
26	How many compounds have space group number 225?	E
27	Among NiAs-type compounds with a hexagonal crystal system, the one with the largest lattice constant c.	M
28	Show B2 compounds whose lattice constant a is 4.0Åor larger.	M
29	Tell me the mean and standard deviation of the lattice constant a for L12-type compounds.	M
30	Tell me the number of registered entries for each prototype.	E

No.	Query (excerpt)	Difficulty
D. Stability and Formation Energy		
31	Show all L12 compounds that constitute the convex_hull.	M
32	Are there any compounds with positive formation energy?	M
33	Show B2 structures that are stable and have negative formation energy.	M
34	Metastable ($E_{\text{hull}} > 0$ and < 0.1 eV/atom) L12 compounds.	M
35	Tell me the fraction of unstable compounds with the NaCl-type structure.	H
36	Are there any L12-type compounds with a band gap greater than 0?	M
37	Among stable L12 compounds, those with a band gap of 0.	M
38	Show all convex_hull distances for compounds in the Ni-Al system.	M
39	Extremely stable compounds with formation energy of -0.5 eV/atom or lower.	M
40	Compare the average energy_above_hull by prototype.	M
E. DFT Calculations and Physical Properties		
41	Are there any entries calculated using functionals other than GGA-PBE?	E
42	Show L12 compounds with a bulk modulus of 200GPa or higher.	H
43	Tell me the TOP3 L12 compounds with the highest shear modulus.	H
44	Show all compounds for which Young's modulus is recorded.	M
45	List compounds with a Poisson's ratio of 0.3 or higher.	M
46	L12 compounds with a B/G ratio of 2 or higher.	VH
47	Are there any compounds identified as elastically unstable in DFT calculations?	M
48	I would like to find compounds with a large c/a in the lattice constants.	M
49	Tell me the bulk modulus and formation energy of Ni3Al.	H
50	Show all L12 compounds with nonzero magnetic moment.	H
F. Electronic Structure, Magnetism, and Thermal Properties		
51	Which L12 compound has the highest DOS at the Fermi level?	H
52	Are there any compounds with a direct band gap?	M
53	Show compounds for which spin-polarized calculations have been performed.	M
54	Show ferromagnetic L12 compounds.	H
55	Which compound has the highest Curie temperature?	M
56	Tell me compounds with a Debye temperature of 500K or higher.	M
57	Show L12 compounds for which thermal conductivity is recorded.	H
58	Are there any compounds with a Grüneisen parameter of 2 or higher?	M
59	Show the L12 compound with the largest magnetic anisotropy energy.	H
60	Find L12 compounds that are both metallic and ferromagnetic.	VH
G. Surfaces, Grain Boundaries, and Defects		
61	Tell me the L12 compound with the lowest surface energy on the (111) plane.	H
62	Show L12 compounds with a work function of 5 eV or higher.	H
63	Are there any compounds in which surface reconstruction occurs?	M
64	Show compounds for which $\Sigma 5$ grain-boundary energy is recorded.	M
65	Which L12 compound has the lowest vacancy formation energy?	H
66	Tell me L12 compounds for which antisite defect information is available.	H
67	Are there any compounds in which B is used as a dopant element?	H
68	Show compounds for which information on interstitial defects is available.	M
69	Compounds that have surface energies for both the (100) and (110) planes.	VH
70	Information on L12 and B2 compounds with large deviations from Vegard's law.	VH
H. Literature, Synthesis, and Applications		
71	Which L12 compounds have been experimentally synthesized?	H
72	Show L12 compounds synthesized by arc melting.	H
73	Show L12 compounds synthesized at temperatures of 1000K or higher.	H
74	Tell me L12 compounds with three or more literature references.	VH
75	Show compounds classified in application areas of high-temperature superalloys.	H

No.	Query (excerpt)	Difficulty
76	Show L12 compounds reported in the literature from 2020 onward.	VH
77	Show compounds for which both experimental and calculated values are available.	H
78	Are there any compounds synthesized by ball milling?	H
79	Show compounds classified as heat-resistant materials.	H
80	Provide a list of literature references for which DOI is recorded.	E
I. Materials Design and Screening		
81	Stable compounds with lattice constants within 0.05Åof Ni3Al and bulk modulus of 150GPa or higher.	VH
82	Rank promising L12 compounds as γ' phase candidates by formation energy.	H
83	Stable L12 compounds containing Co with bulk modulus of 180GPa or higher and high Debye temperature.	VH
84	Stable L12 compounds with lattice constant 3.56Å±0.03Åand shear modulus of 70GPa or higher.	VH
85	Show all ferromagnetic and elastically stable L12 compounds.	VH
86	Show L12 compounds with experimental synthesis records and stability by DFT.	VH
87	Compounds with a bulk modulus higher than Ni3Al and a hull distance of 0.01 or less.	VH
88	Aggregate stable compounds with two elements by prototype.	H
89	L12 compounds with a low Poisson's ratio (<0.25) predicted to have low ductility.	H
90	Stable L12 compounds with low surface energy (<1.5 J/m^2).	VH
J. Comparison and Statistics		
91	Compare the fraction of stable compounds between L12 and B2.	H
92	Tell me the average bulk modulus by prototype.	H
93	Aggregate the number of L12 compounds for each A-site element.	H
94	Is there a correlation between lattice constant and formation energy? Use all L12 compound data.	M
95	Is there a difference in the average bulk modulus between stable and unstable L12 compounds?	VH
96	Tell me the TOP10 chemical systems by number of entries.	E
97	Show the number of entries by DFT calculation method.	E
98	Distribution of band gaps for L12-type compounds. Separate those that are 0 and those that are not.	H
99	Data for displaying the distribution of lattice constant a as a histogram in 0.1Åincrements.	H
100	Breakdown of the number of stable/unstable/metastable cases for all prototypes.	H

Table S7 shows the accuracy by category.

Table S7: Accuracy by category for the independently designed queries.

Category	Content	Correct/Total	Accuracy
A	Basic search	7/10	70.0%
B	Composition and elements	8/10	80.0%
C	Structure and lattice constants	7/10	70.0%
D	Stability and formation energy	6/10	60.0%
E	DFT calculations and physical properties	9/10	90.0%
F	Electronic structure, magnetism, and thermal properties	10/10	100.0%
G	Surfaces, grain boundaries, and defects	10/10	100.0%
H	Literature, synthesis, and applications	8/10	80.0%
I	Materials design and screening	8/10	80.0%
J	Comparison and statistics	4/10	40.0%
Overall		**77/100**	**77.0%**

Categories F (electronic structure, magnetism, and thermal properties) and G (surfaces, grain boundaries, and defects) achieved 100%. With the introduction of the SUPERSET detection repair loop, category I (materials design and screening) improved substantially from the previous 30% to 80%. This improvement was largely due to the automatic detection and correction of queries with insufficient conditions. The 40% accuracy in category J (comparison and statistics) remains an issue, indicating room for improvement in processing prototype-crossing aggregation and compound GROUP BY conditions.